

基于物理约束引导与智能优化的无人机烟幕干扰弹投放策略研究

摘要

烟幕干扰弹凭借成本低、效费比高的优势，在战场目标遮蔽与敌方制导系统干扰中占据关键地位。本文以无人机投放烟幕干扰弹对抗导弹袭击为背景，围绕“最大化真目标累计有效遮蔽时长”这一核心目标，构建了多维度动力学模型与分层优化框架，优化无人机投弹策略，系统地解决了多个场景下的投放策略优化问题。

针对问题一，已知无人机飞行参数与烟幕弹投爆时序，建立了导弹运动模型、无人机运动模型、烟幕干扰弹运动模型及烟幕云团运动模型。对于需要计算的有效遮蔽时长，创新地提出了**双锥遮蔽判定模型**，通过高密度采样真目标上下表面圆环点，检测导弹与采样点连线是否被烟幕球体遮挡，结合时间步长扫描法即可高效计算有效遮蔽时长，单次运行时长平均压缩至 **0.027s**。最终求得该场景下总遮蔽时长为 **1.39 秒**。

针对问题二，在单无人机投放单枚烟幕弹的优化场景中，以无人机飞行角度、速度、烟幕弹投放时刻与起爆延时为决策变量，构建单目标优化模型。采用**动态参数搜索的粒子群优化算法**，以遮蔽时长为适应度函数，在参数约束空间内迭代寻优；同时引入遗传算法交叉验证，最终得到最优参数组合。（飞行角度 **7.18°**、速度 **95.73 m/s**），对应最大有效遮蔽时长 **4.59 秒**，两种算法结果一致性验证了优化解的可靠性。

针对问题三，扩展至单无人机投放三枚烟幕弹场景，考虑多烟幕云团共存特性，改进遮蔽判定模型为**多云团并集遮蔽判定**；以无人机飞行参数与三枚弹的投爆时序为 8 维决策变量，在投弹间隔不小于 1 秒的约束下，采用粒子群优化算法求解，最终实现 **6.41 秒**的总遮蔽时长。

针对问题四，在三架无人机各投一枚烟幕干扰弹的协同场景中，由于决策变量更加复杂，我们创新性地采用**物理约束引导粒子群初始化以缩小解空间范围、退火策略粒子群全局优化**这一两阶段策略：先通过**无人机可达域、烟幕弹下落动力学约束**筛选候选起爆点，结合张角判别法裁剪无效解；再以三架无人机的 12 维飞行与投爆参数为决策变量，引入模拟退火机制改进粒子群算法，避免局部最优，最终实现 **11.56 秒**总遮蔽时长，且基于引导的粒子群初始化，**能快速找到最优解，大幅度提高性能**。

针对问题五，在五架无人机遮蔽三枚导弹的复杂场景中，面对指数级增长的搜索空间，受第四问启发，对其作出进一步改进，提出了**物理约束引导的启发式候选生成、航向-速度聚类降维与贪心选优、粒子群全局优化**的三阶段求解策略。定义多导弹总有效遮蔽时长为目标函数，通过区间合并计算单导弹遮蔽时长，结合无人机速度、投弹间隔等约束，在简化多弹协同复杂度的同时，实现多目标、多资源的全局最优配置。在**两分钟**的运行时间内，实现了 **34.00 秒**的总有效遮蔽时长。

关键词：无人机投弹策略的约束优化模型 物理约束引导 启发式搜索 解空间优化 双锥遮蔽判定模型 粒子群算法 模拟退火算法

一、问题重述

1.1 问题背景

烟幕干扰弹作为现代特种弹药的重要组成部分，依靠化学燃烧或爆炸，将烟雾或气溶胶散布在目标前方的指定空域，形成遮蔽以干扰敌方观测和制导系统。其核心优势在于成本低、效费比高，在战场遮蔽、目标干扰和战术掩护中发挥着关键作用。现在考虑使用无人机来完成烟幕干扰弹的投放策略问题。在此任务中，可将策略设计抽象为以无人机航迹、飞行速度、投放位置和起爆时机为决策变量、以目标的累计有效遮蔽时间最大化（可不连续）为目标的优化问题。

1.2 需要解决的问题

已知导弹、无人机、真假目标的坐标，无人机受领任务后可瞬时调整方向、以 $70\sim 140\text{ m/s}$ 的速度等高度匀速直线飞行，每架无人机投放两枚烟幕干扰弹间隔至少 1 s ，干扰弹脱离后在重力下进行运动，烟幕云团以 3 m/s 的速度匀速下沉，在起爆 20 s 内，云团中心 10 m 范围内可提供有效遮蔽，袭击导弹飞行速度 300 m/s ，飞行方向直指假目标，真目标为高 10 m 、半径 7 m 的圆柱体。要求建立数学模型解决以下问题：

1.2.1 问题一

在给定无人机 $FY1$ 飞行速度与方向、投放烟幕干扰弹和起爆时间的情况下，计算干扰弹对导弹 $M1$ 的有效遮蔽时长。

1.2.2 问题二

在只有一架无人机 $FM1$ 投放一枚烟幕干扰弹干扰 $M1$ 的情况下，确定对导弹 $M1$ 的遮蔽时间最长时， $FY1$ 的飞行方向、飞行速度、烟幕干扰弹投放点和烟幕干扰弹起爆点的取值。

1.2.3 问题三

在一架无人机 $FY1$ 投放三枚烟幕干扰弹干扰 $M1$ 的情况下，规划每一枚干扰弹的投放策略，以最大化对 $M1$ 的干扰效果，并保存结果到规定文件中。

1.2.4 问题四

在三架无人机 $FY1$ 、 $FY2$ 、 $FY3$ 各投放一枚烟幕干扰弹干扰 $M1$ 的情况下，规划每架无人机的投放策略，以最大化对导弹 $M1$ 的总有效遮蔽时长，并保存结果到规定文件中。

1.2.5 问题五

在五架无人机（每架至多投放三枚烟幕干扰弹），对三枚导弹 $M1$ 、 $M2$ 、 $M3$ 进行干扰的情况下，设计烟幕干扰弹的投放策略，并保存结果到规定文件中。

二、问题分析

2.1 问题一的分析

问题一已知无人机 $FY1$ 飞行速度与方向、投放烟幕干扰弹和起爆时间，要求得到干扰弹对导弹 $M1$ 的有效遮蔽时长。首先根据条件建立导弹运动模型、无人机运动模型、烟幕干扰弹投放与运动模型和烟幕云团运动模型。对于有效遮蔽时长的计算，最关键的是建立遮蔽判定模型：我们考虑导弹与目标圆柱体上下表面圆环的连线是否与云团相交，

若边界条件成立，则判定为有效遮蔽，据此建立了双锥遮蔽判定模型，编写代码，通过对每个时间步求解，得出第一问。

2.2 问题二的分析

问题二要求改变 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点和烟幕干扰弹起爆点，找到一组最优取值，使干扰弹对导弹 M1 的有效遮蔽时间最长，是一个典型的优化问题。飞行方向转化为 XY 平面的角度，烟幕干扰弹投放点和烟幕干扰弹起爆点转化为投放时刻和起爆延时。在问题一模型的基础上，我们进一步考虑无人机 FY1 的飞行参数与烟幕干扰弹的投爆时机，有效遮蔽时长的计算仍依赖双锥遮蔽判定模型。

本题我们使用了两种方法进行求解：

1. 引入**动态参数搜索的粒子群优化算法**，以遮蔽总时长为适应度函数，在参数空间[飞行角度，飞行速度，投放时刻，起爆延时]内搜索最优解。算法通过迭代更新粒子群的位置与速度，不断逼近使遮蔽时长最大的参数组合。最终得到 FY1 的最优飞行方向与速度、烟幕干扰弹的投放点和起爆点，从而实现最长的有效遮蔽。

2. 以“有效遮蔽时长”为目标函数，将投放时刻、起爆延时等作为决策变量，引入**遗传算法**进行全局优化。通过种群初始化、选择、交叉、变异等操作迭代更新参数，最终得到能够实现最长遮蔽时长的最优投放策略，并在仿真中加以验证。

2.3 问题三的分析

问题三在问题二的基础上，进一步要求我们在 FY1 能投掷三枚烟幕干扰弹的情况下，寻求最佳策略，最大化导弹 M1 的被遮蔽时长。由于会出现多烟幕干扰弹共存的情况，我们首先**修正了我们的遮蔽判别模型**，使其能够适用于多烟幕干扰弹共存的场景。然后我们列出了决策变量，目标函数，约束条件，将其视为一个约束优化问题，通过建立与第二问类似的**粒子群优化算法**求得最终结果。

2.4 问题四的分析

为方便公式推导，我们以 FY1, FY2, FY3 三架无人机各自的飞行方向、飞行速度、投弹时间、引爆时间作为决策变量，最大化导弹 M1 被烟雾遮挡的时间，建立优化问题。与问题二相比，模型决策空间更加复杂，为了避免陷入局部最优解，我们将优化分为启发式**策略空间搜索**与**智能算法优化**两个阶段执行。

首先，我们进行**物理约束引导下的解空间优化**。在离散空间内近似枚举导弹 M1 运行时间，基于无人机可达域约束与烟幕干扰弹下落动力学约束，搜索出一系列初步的引爆点。再基于真目标的张角与云球的张角，对候选起爆点再次筛选，并根据问题三中得到的遮蔽判定模型，判断烟幕干扰弹在引爆后 20s 内对真目标的遮蔽情况，确定最终的可行起爆点。然后搜索 FY1, FY2, FY3 所有起爆点中的组合，最大化导弹 M1 的被遮挡时长，并依据无人机与烟幕干扰弹的动力学方程反解出其对应的决策变量。最后**依据反解出的较优决策变量初始化粒子群算法**，并在粒子群更新过程中**嵌入模拟退火算法**作为优化，求得最终结果。

2.5 问题五的分析

问题五在前四问的基础上扩展为多无人机、多烟幕、多导弹的复杂协同优化问题，搜索空间呈指数级增长。我们引入引入区间合并计算导弹的有效遮蔽时长，并以多个导弹的总有效遮蔽时长作为目标函数。为降低复杂度，本问在问题四的基础上进一步改进

了解空间优化算法，形成了物理约束引导的启发式候选生成、航向-速度聚类降维与贪心选优、粒子群全局优化” 的三阶段求解框架，大大提升了计算效率和优化精度。

三、模型假设

1. 忽略空气阻力和摩擦力带来的影响。
2. 烟幕干扰弹脱离无人机后，水平方向速度与无人机大小方向相同，竖直方向仅受重力。
3. 云团呈球状，不考虑烟幕的扩散。云团以 3m/s 的速度匀速下沉，水平方向无速度
4. 真目标视为半径 7m、高 10m 的标准圆柱。

四、符号说明

表1 符号说明

符号	含义
v_m, v_f, v_c	导弹，无人机，烟幕云团的运动速度
v_{s-xy}	烟幕干扰弹水平方向运动速度
$\hat{d}$	单位方向向量
t	运动时间
t_{rel}	烟幕干扰弹投放时刻
t_e	烟幕干扰弹起爆延时
t_{eff}	云团遮掩有效时间
t_{all}	导弹被有效遮蔽的总时长
(x, y, z)	三位坐标系上的坐标
M	导弹的位置
F	无人机的位置
P	烟幕干扰弹的位置
C	烟幕云团中心的位置
T_a	假目标的位置
T	真目标中心位置
R	烟幕云团半径
P_{xy}	P 在 x-y 平面上的二维投影点坐标
P_x	P 的 x 坐标

五、模型的建立与求解

5.1 问题一模型建立与求解

5.1.1 模型的建立

(1)运动模型的建立

导弹运动模型：

根据题目，导弹 $M1$ 以 $v_m = 300m/s$ 的速度直指假目标飞行。由导弹初始位置 $M_0 = (20000, 0, 2000)$ ，假目标位置为 $T_0 = (0, 0, 0)$ ，可求出单位方向向量

$$\hat{d} = \frac{T_f - M_0}{\|T_f - M_0\|}$$

其中 $\|T_f - M_0\|$ 表示欧几里得距离。导弹沿方向 $\hat{d}$ 以恒定速度 v_m 飞行，则任意时间 t 的位置为：

$$M(t) = M_0 + v_m \hat{d} t$$

无人机运动模型：

我们已知无人机受领任务后，可根据需要瞬时调整飞行方向，然后以 $70 \sim 140 m/s$ 的速度等高度匀速直线飞行，因此无人机运动只需考虑 XY 平面的运动。设无人机在 XY 平面以速度 v_f 沿飞行角度 θ （以 X 轴正方向为 0° ，逆时针为正）匀速飞行，无人机初始位置为 $F_0 = (x_0, y_0, z_0) = (17800, 0, 1800)$ ，则任意时间 t 的位置为：

$$D(t) = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} + v_f t \begin{pmatrix} \cos \theta \\ \sin \theta \\ 0 \end{pmatrix} = \begin{pmatrix} x_0 + v_f \cos \theta t \\ y_0 + v_f \sin \theta t \\ z_0 \end{pmatrix}$$

问题一中无人机 $FY1$ 以 $v_f = 120 m/s$ 的速度朝向假目标方向飞行，就是沿 x 轴负方向水平飞行，此时 $\theta = 180^\circ$ 。带入可得问题一中 $FY1$ 在任意时间 t 的位置为：

$$F(t) = (x_0 - v_f t, y_0, z_0)$$

烟幕干扰弹投放与运动模型：

无人机在 t_{rel} 秒时投放烟幕干扰弹，间隔 t_e 起爆，则投放位置为：

$$P_{rel} = F(t_{rel}) = (x_r, y_r, z_r)$$

无人机在 XY 平面匀速运动，投放时烟幕干扰弹初速度与无人机速度相同，即

$$v_{s-xy} = v_f$$

烟幕干扰弹的 Z 方向初速度为 0 （自由下落）。重力加速度为 $g = 9.8m/s^2$ ，从投放到起爆之间的时间间隔为 $\Delta t = t - t_{rel}$ ，则烟幕干扰弹位置为：

$$P(t) = \left(x_r + v_f \Delta t, y_r, z_r - \frac{1}{2} g (\Delta t)^2 \right), t \in [t_{rel}, t_{rel} + t_e]$$

烟幕云团运动模型：

起爆后，烟幕云团仅以 $v_c = 3 m/s$ 的速度匀速垂直下沉，水平位置保持不变。设烟幕干扰弹起爆时间 $t_c = t_{rel} + t_e$ ，起爆时位置为 $C_e = P(t_c) = (x_e, y_e, z_e)$ ，则任意时间 $t \geq t_c$ 的烟幕中心位置：

$$C(t) = (x_e, y_e, z_e - v_c(t - t_c))$$

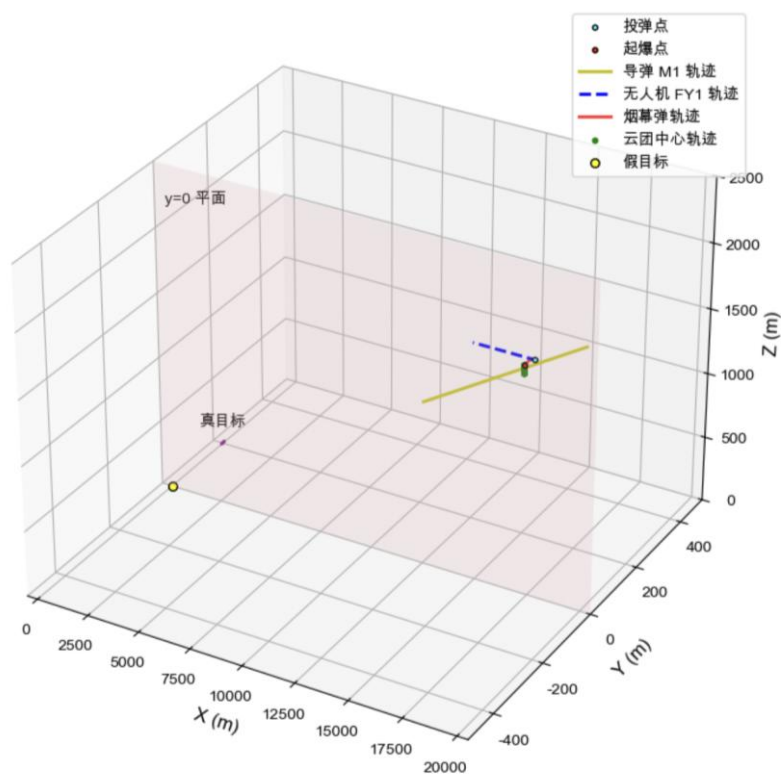


图1 无人机、导弹与烟幕干扰弹三维轨迹图

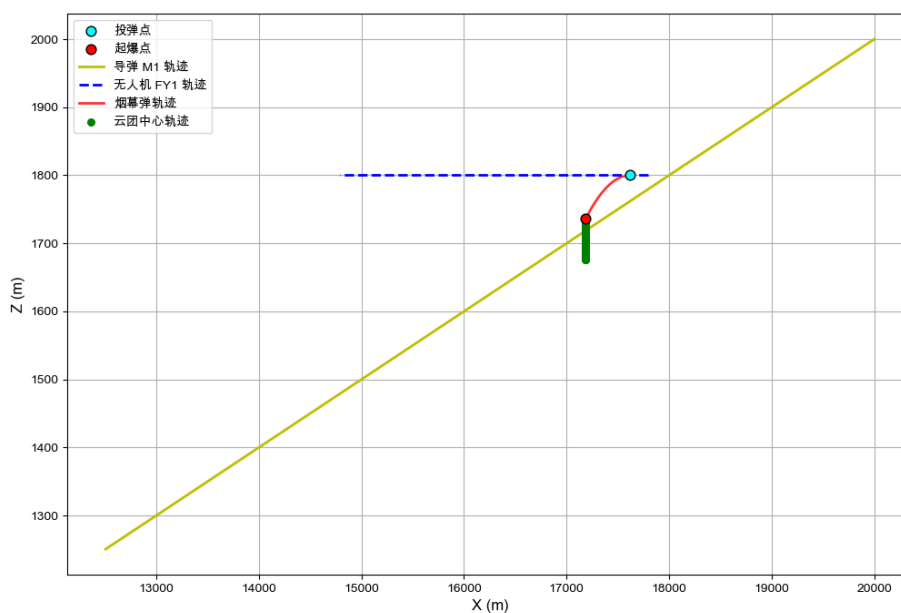


图2 无人机、导弹与烟幕干扰弹平面轨迹图

(2)双锥遮蔽判定模型的建立

首先我们明确有效遮蔽的定义：烟幕云团在目标前方特定空域形成遮蔽，干扰敌方导弹。因此遮蔽判定模型就是判定在导弹飞向假目标的过程中，烟幕云团是否能在导弹的视线范围内完全遮挡真实目标，使导弹无法探测到真实目标。

考虑到，如果导弹与目标圆柱体上下表面圆环的连线均不与云团相交，则导弹与圆柱体表面任意一点连线也都不与云团相交，因此可将判定化简为只需判断导弹与真目标

上下表面圆环连线是否全被遮蔽，无需计算圆柱表面的所有点，大大降低了计算量（将程序平均运行时间缩短至0.027秒）。我们选择建立几何遮蔽判定模型来判断云团是否有效遮蔽导弹，并计算遮蔽的持续时间。

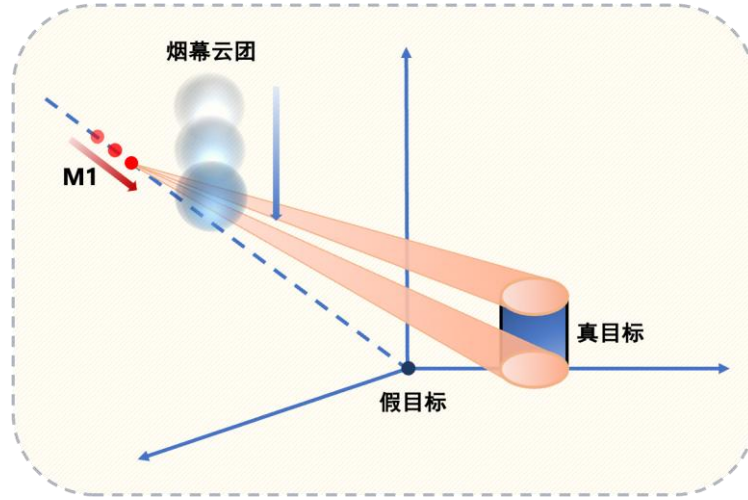


图3 双锥遮蔽判定模型示意图

首先将真目标视为半径 $7m$ 、高 $10m$ 的标准圆柱，建立其坐标模型。通过对圆柱体上、下表面边界圆环高密度采样（若干点 $\{T_i\}$ ），近似表示真目标上、下表面的边界。

对每个目标点 T_i ，判断其到导弹的连线是否与烟幕球体相交。目标点到导弹位置的直线段的参数方程为：

$$L(s) = T_i + s(M(t) - T_i), \quad 0 \leq s \leq 1$$

其中 T_i 为真目标表面上的某个采样点， $M(t)$ 为时刻 t 导弹的位置。当 s 从0变到1时， $L(s)$ 就描绘出这条直线段上的所有点。如果存在某个 $s \in [0,1]$ ，使得 $L(s)$ 落在烟幕球体内，则从该目标点看导弹的视线被烟幕挡住。

将烟幕云团视为一个标准球体，其球体方程为：

$$|x - C(t)|^2 \leq R^2$$

其中 $C(t)$ 是时刻 t 烟幕中心位置， $R = 10m$ 为云团遮挡半径。

将 $L(s)$ 带入球体方程中的 x ，转化为一元二次方程：

$$as^2 + bs + c = 0$$

若存在实根且 $s \in [0,1]$ ，说明视线被烟幕遮挡。

只有对真目标所有采样点 T_i ，都满足“被烟幕遮挡”，我们才认为烟幕干扰弹对导弹进行了有效遮蔽。

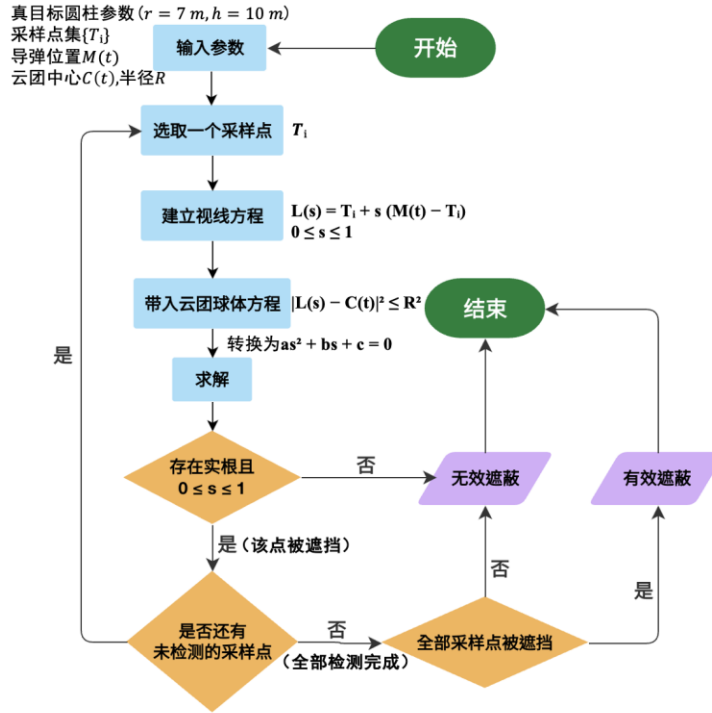


图4 判定流程图

5.1.2 模型的求解

求解基本逻辑是从烟幕干扰弹爆炸时刻开始，逐步模拟导弹、真目标和烟幕云团的相对位置；判断在某一时刻云团是否能完全遮蔽真目标；记录有效遮蔽的所有时间段，求和得到总遮蔽时长。

逐步扫描求解

这里我们采用逐步扫描的方法来求解总遮蔽时长 t_{all} ，步骤如下：

首先密集采样真目标上、下表面圆环上的点，将圆环离散为若干点，存为 $\{T_i\}$ 。将 t 的初始值设为烟幕干扰弹起爆时间 t_c ，由云团有效时间 $t_{eff} = 20\text{s}$ ，可得 $t \in [t_c, t_c + t_{eff}]$ 。设定步长 λ 和最长模拟时刻 $t_{max} = t_c + t_{eff}$ ，从 t_c 开始逐步推进时间，计算导弹位置 $M(t)$ ，烟幕云团中心 $C(t)$ 。如果时间已经超过烟幕最大有效时长：若还在遮蔽状态，则直接封闭这一段区间，否则跳出循环。

检查每个采样点与导弹连线是否与烟幕球体相交，若所有目标点均被遮蔽，则该时刻为有效遮蔽。如果此刻遮蔽有效而之前未开始，则记录起点 t_1 ；如果此刻遮蔽失效而之前在遮蔽，则记录终点 t_2 。将 (t_1, t_2) 存入遮蔽区间列表 $t_{effective}$ 。如果循环结束时还在遮蔽状态，则把最后一个区间补上。最后计算总有效遮蔽时长 $t_{all} = \sum(t_2 - t_1)$ 。

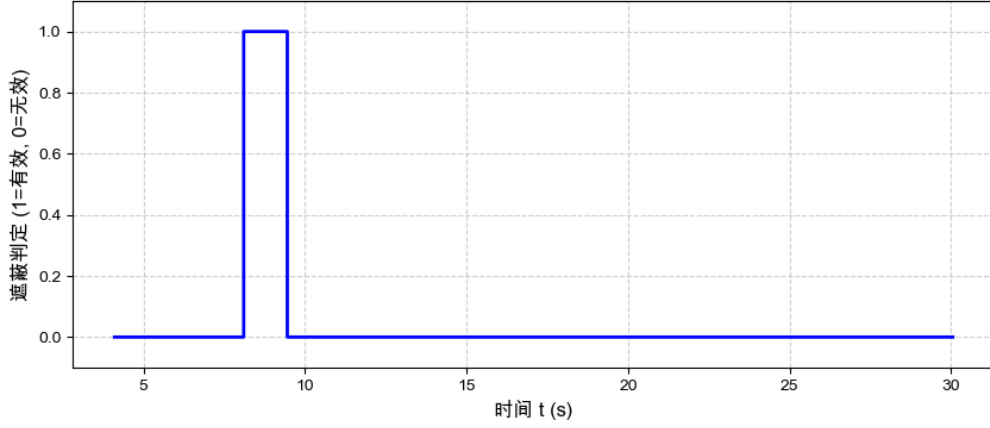


图5 烟幕云团干扰有效遮蔽判定图

以伪代码形式呈现如下：

逐步扫描时间求解总遮蔽时长 t_{all} 伪代码
<pre> $t \leftarrow t_c$, $\lambda = 0.01$, $t_{max} \leftarrow t_c + t_{eff}$, $t_{effective} \leftarrow \emptyset$, $t_1 \leftarrow None$ while $t \leq t_{max}$ $M_{pos} \leftarrow M(t)$, $C_{center} \leftarrow C(t)$ $is_{effective} \leftarrow True$ for $T_i \in T_i$ if 判定为无效遮蔽 $is_{effective} \leftarrow False$ break end end if $is_{effective} \wedge t_1 = None$ $t_1 \leftarrow t$ else if $\neg is_{effective} \wedge t_1 \neq None$ $t_2 \leftarrow t$ $t_{effective}.append((t_1, t_2))$ $t_1 \leftarrow None$ end $t \leftarrow t + \lambda$ end if $t_1 \neq None$ $t_{effective}.append((t_1, t_{max}))$ end $t_{all} \leftarrow \sum(t_2 - t_1)$ </pre>

得出结果

结果输出为遮蔽区间列表 $t_{effective} = [(t_1, t_2), \dots]$ ，以及总遮蔽时长 $t_{all} = \sum(t_2 - t_1)$ 。

表2 问题一的求解结果

输出	具体数据
有效遮蔽时间段	[(8.059999999999937,9.449999999999907)]
总有效遮蔽时长	1.390 秒

5.2 问题二模型建立与求解

已知无人机 $FY1$ 初始位置 F_0 ，导弹 $M1$ 初始位置 M_0 和速度 v_m ，目标圆柱体参数，烟幕云团下沉速度 v_c 、云团半径 R 和云团遮掩有效时间 t_{eff} 。选择无人机 $FY1$ 的飞行方向 θ 、飞行速度 v_f 、烟幕干扰弹投放时刻 t_{rel} 与起爆延时 t_e ，使得导弹被有效遮蔽的总时长 t_{all} 最大化。

5.2.1 单烟单弹优化模型的建立

首先沿用问题一的导弹运动模型 $M(t)$ 、无人机运动模型 $F(t)$ 、烟幕干扰弹运动模型 $P(t)$ 烟幕云团运动模型 $C(t)$ 、双锥遮蔽判定模型。

在本问题中，优化的目标是**最大化**烟幕干扰弹对导弹的有效遮蔽时长。因此我们需要建立一个**单目标优化模型**。首先设定无人机的优化决策变量：

$$x = [\theta, v_f, t_{rel}, t_e]$$

其中无人机飞行角度 $\theta \in [0^\circ, 360^\circ]$ ，无人机飞行速度 $v_f \in [70, 140]$ ，烟幕干扰弹投放时刻 $t_{rel} \in [0, 70]$ ，烟幕干扰弹起爆延时 $t_e \in [0, 20]$ 。

针对给定参数组合 x ，通过导弹、无人机及烟幕干扰弹运动学模型，计算目标在导弹飞行过程中的可见性，并得到遮蔽持续时间：

$$T_{sd} = f(\theta, v_f, t_{rel}, t_e)$$

因此，我们确定**遮蔽持续时间** T_{sd} 的**单目标优化模型**为：

$$\begin{aligned} \max_{\theta, v_f, t_{rel}, t_e} \quad & T_{sd} = f(\theta, v_f, t_{rel}, t_e) \\ \text{s.t.} \quad & \begin{cases} 0^\circ \leq \theta \leq 360^\circ \\ 70 \leq v_f \leq 140 \\ 0 \leq t_{rel} \leq 70 \\ 0 \leq t_e \leq 20 \end{cases} \end{aligned}$$

5.2.2 单烟单导弹优化模型的求解

为了在优化过程中兼顾全局搜索与局部开发、提升收敛速度与精度并降低早熟收敛风险，我们结合了**动态参数搜索**和**粒子群优化算法**。每个粒子的位置向量 $x_i = [\theta, v_f, t_{rel}, t_e]$ ，其搜索空间受到物理和任务条件的约束，粒子群在参数空间中迭代搜索，逐步接近最优解。在粒子群优化算法中，适应度函数的数值越大，说明该参数组合下烟幕干扰弹产生的干扰效果越优，个体的适应度越高。因此，将适应度函数定义为：

$$F(x) = T_{sd}$$

即适应度函数的值等于对应参数下的有效遮蔽时长。

STEP1 初始化

首先在参数边界约束内随机生成粒子群的初始位置向量与速度向量：

$$x_i^{(0)} = (\theta_i^{(0)}, v_i^{(0)}, t_{rel_i}^{(0)}, t_{e_i}^{(0)}), i = 1, 2, \dots, N,$$

$$v_i^{(0)} = (v_{\theta,i}^{(0)}, v_{v,i}^{(0)}, v_{t_{rel},i}^{(0)}, v_{t_e,i}^{(0)})$$

其中 N 为粒子数。随后计算每个粒子的适应度函数：

$$f(x_i^{(0)}) = T_{sd}(\theta_i^{(0)}, v_i^{(0)}, t_{rel_i}^{(0)}, t_{e_i}^{(0)})$$

并确定个体最优位置 p_i^* 和全局最优位置 g^* 。

STEP2 迭代优化

在每一个迭代过程中，粒子的位置和速度按照以下公式更新：

$$v_i^{(k+1)} = \omega^{(k)} v_i^{(k)} + c_1 r_1 (p_i^* - x_i^{(k)}) + c_2 r_2 (g^* - x_i^{(k)})$$

$$x_i^{(k+1)} = x_i^{(k)} + v_i^{(k+1)}$$

其中 $\omega^{(k)}$ 为惯性权重，初始惯性权重 $\omega_{max} = 0.9$ ，终止时惯性权重 $\omega_{min} = 0.4$ ，并随着迭代次数线性递减：

$$\omega^{(k)} = \omega_{max} - \frac{k}{K_{max}} (\omega_{max} - \omega_{min})$$

c_1, c_2 为学习因子； $r_1, r_2 \sim U(0,1)$ 为随机数。

随后重新计算粒子适应度 $f(x_i^{(k+1)})$ ，并更新个体最优解 p_i^* 及全局最优解 g^* 。

STEP3 收敛与最优解输出

当达到最大迭代次数 K_{max} 最优解收敛时，算法终止。此时得到的最优参数组合为：

$$x^* = (\theta^*, v^*, t_{rel}^*, t_e^*)$$

计算其对应的最大有效遮蔽时长：

$$T_{sd}^* = f(x^*)$$

最终，输出无人机飞行角度 θ^* 、飞行速度 v^* 、烟幕干扰弹投放时刻 t_{rel}^* 、起爆延时 t_e^* 及最大遮蔽时长 T_{sd}^* ，作为问题二的优化解。

求解结果

本题中，我们选取粒子数量 $N = 50$ ，最大迭代次数 $K_{max} = 80$ 进行迭代。为确保优化过程的可重复性，设置随机种子 $seed = 1$ ，个体学习因子与群体学习因子分别取 $c_1 = c_2 = 1.49$ 。

表3 问题二的求解结果

输出	具体数据
无人机飞行角度 θ (度)	7.18°
无人机飞行速度 v (m/s)	95.733
烟幕干扰弹投放位置	[17816.68, 2.1, 1800]
烟幕干扰弹起爆位置	[17894.17, 1.2, 1796.74]
总有效遮蔽时长(t)	4.590

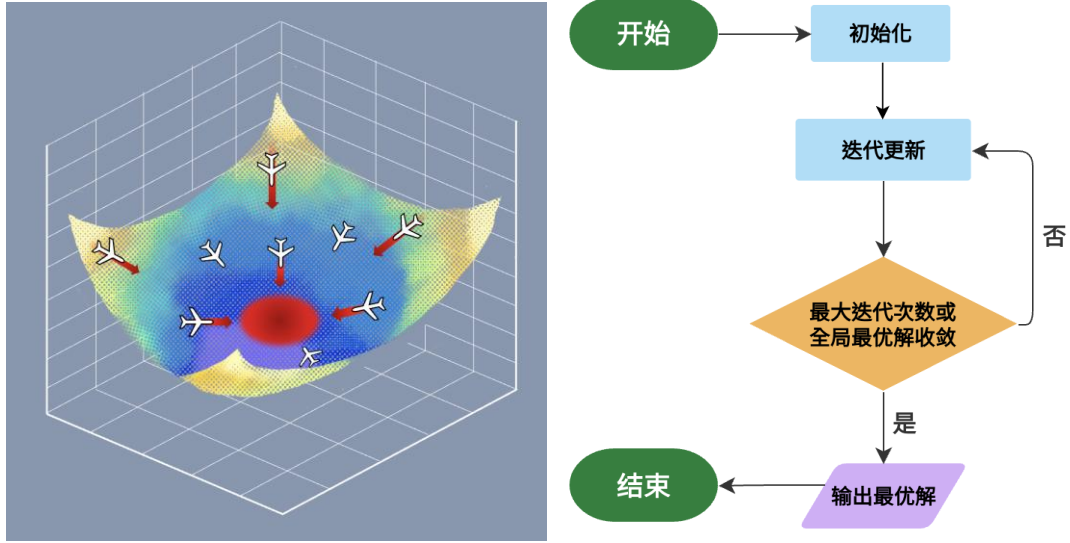


图6 粒子群优化示意图和算法流程图

5.2.3 遗传算法优化验证

为了验证粒子群优化算法结果的准确性，我们还使用遗传算法对同一优化问题进行求解。遗传算法作为一种经典的群体智能优化方法，通过选择、交叉和变异等操作不断进化群体，最终获得最优解。其求解流程如图7所示。

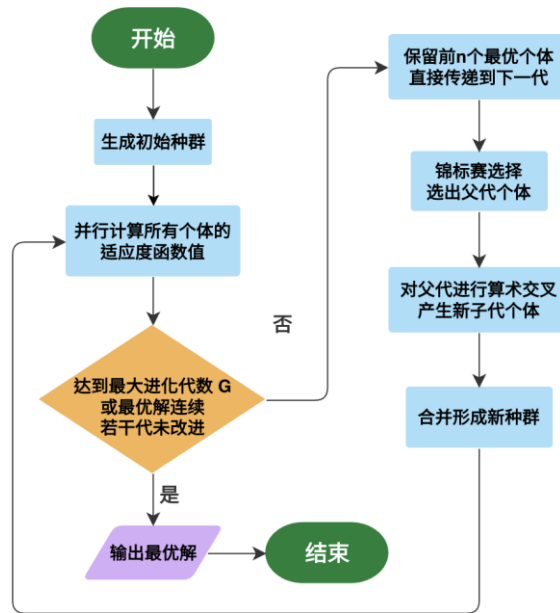


图7 遗传算法优化流程图

计算得到最大遮蔽时长为**4.59s**，与动态参数粒子群优化算法结果一致，进一步验证了前述算法的正确性与可靠性。

5.3 问题三模型建立与求解

已知无人机 $FY1$ 初始位置 F_0 ，导弹 $M1$ 初始位置 M_0 和速度 v_m ，目标圆柱体参数，烟幕云团下沉速度 v_c 、云团半径 R 和云团遮掩有效时间 t_{eff} 、两枚烟幕干扰弹**至少间隔时间**

t_{min} 。选择无人机FY1的飞行方向 θ 、飞行速度 v_f 、烟幕干扰弹投放时刻 t_{rel} 与起爆延时 t_e ，使得导弹被有效遮蔽的总时长 t_{all} 最大化。

5.3.1 一机多烟单导弹优化模型的建立

一枚烟幕云团在场上最多存在20s，在第三问的情况中，存在多烟幕干扰弹共存的问题。为了判断导弹是否被遮挡，我们要建立一个新的判断遮蔽函数，即借助第一问中建立的遮蔽判断模型，我们检测导弹与圆柱上下底面圆周的采样点的连线是否均穿过烟幕云团的并集。若是，则判断为遮蔽成功，反之则为失败。

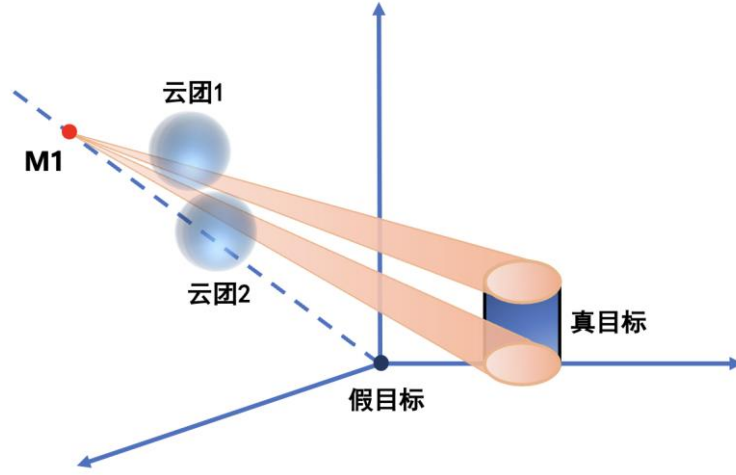


图8 优化的遮蔽判定方法示意图

(2) 决策变量

决策变量 x 是一个包含了，无人机速度，无人机航向，第一枚烟幕干扰弹投掷时间，第二枚烟幕干扰弹投掷时间，第三枚烟幕干扰弹投掷时间，第一枚烟幕干扰弹引爆时间，第二枚烟幕干扰弹引爆时间，第三枚烟幕干扰弹引爆时间的8维向量。

$$x = (v, \theta, t_{rel_1}, t_{rel_2}, t_{rel_3}, t_{e_1}, t_{e_2}, t_{e_3})$$

(3) 目标函数

优化目标是寻求三枚烟幕干扰弹对M1的总遮蔽时长最长。针对每一枚烟幕干扰弹 P_i ，都可以单独定义遮蔽时长 t_{all_i} ：

$$t_{all_i} = \int_{t_{start_i}}^{t_{end_i}} J(t) dt$$

总的目标函数定义为区间 (t_{start_i}, t_{end_i}) ， $i = 1, 2, 3$ 的并集的长度。

$$J = \left| \bigcup_{i=1}^3 [t_{start,i}, t_{end,i}] \right|$$

(4) 约束条件

无人机速度约束：

$$v_f \in [70, 140] \text{ m/s}$$

投弹-引爆时序约束：

$$t_e \geq t_{rel} \geq 0$$

投弹间隔约束：

$$t_{rel_{i+1}} - t_{rel_i} \geq 1, i = 1, 2$$

5.3.2 一机多烟单导弹优化模型的求解

该模型的求解与第二问较为相似。设定 $N = 150$, $K_{\max} = 80$, $c_1 = c_2 = 1.49$, $\text{seed} = 1$, 对给定粒子 x_i , 按时间步 Δ 模拟: 导弹轨迹、三枚烟幕弹投放、下落、爆炸后云团轨迹, 用上述模型中改进的“并集遮蔽判定函数”判断每个时刻导弹是否被遮蔽, 即可求得优化结果。根据题目要求, 将特定参数保存到 result1.xlsx。

求解结果

表4 问题三的求解结果

输出	具体数据
无人机飞行角度 θ (度)	8.696976823°
无人机飞行速度 v (m/s)	110.2806971
烟幕干扰弹 1 投放位置	[17800,0,1800]
烟幕干扰弹 1 起爆位置	[17800.17098,0.026154369,1799.999988]
烟幕干扰弹 1 有效干扰时长	2.57
烟幕干扰弹 2 投放位置	[17909.01267,16.67539674,1800]
烟幕干扰弹 2 起爆位置	[17909.01267,16.67539674,1800]
烟幕干扰弹 2 有效干扰时长	3.76
烟幕干扰弹 3 投放位置	[25430.88725,1167.277772,1800]
烟幕干扰弹 3 起爆位置	[25989.53241,1252.732327,1671.319077]
烟幕干扰弹 3 有效干扰时长(s)	0
总有效遮蔽时长(s)	6.41

5.4 问题四模型建立与求解

已知无人机 $FY1$ 、 $FY2$ 、 $FY3$ 初始位置 F_0 、 F_1 、 F_2 , 导弹 $M1$ 初始位置 M_0 和速度 v_m , 目标圆柱体参数, 烟幕云团下沉速度 v_c 、云团半径 R 和云团遮掩有效时间 t_{eff} 。选择无人机 $FY1$ 的飞行方向 θ 、飞行速度 v_f 、烟幕干扰弹投放时刻 t_{rel} 与起爆延时 t_e , 使得导弹被有效遮蔽的总时长 t_{all} 最大化。

沿用问题一的导弹运动模型 $M(t)$ 、无人机运动模型 $F(t)$ 、烟幕干扰弹运动模型 $P(t)$ 烟幕云团运动模型 $C(t)$ 、遮蔽判定模型。

5.4.1 离散空间候选引爆点的确定

(1) 无人机可达域约束

从俯视角度观察, 当给定运动时间 t 的时候, 无人机 FYi 可达域 V_i 是一个内环半径为 $v_{\min}t$, 外环半径为 $v_{\max}t$ 的圆环域, $v_{\min}$ 和 $v_{\max}$ 为无人机速度 v_f 的极限值。由于水平方向上速度不变, 无人机的可达域同样是烟幕干扰弹的可达域。因此在 t 时刻无人机 FYi 的可达域 V_i 为:

$$V_i(t) = \{(x, y) \mid v_{\min}t \leq \sqrt{(x - x_i)^2 + (y - y_i)^2} \leq v_{\max}t\}$$

考虑导弹 $M(t) = (x_m(t), y_m(t), z_m(t))$ 与真目标中心 $T(0, 200, 5)$ 的连线在 x - y 水平平

面上的投影:

$$L_{MT}(t) = \{(1 - \lambda)(x_m(t), y_m(t)) + \lambda(0, 200) \mid 0 \leq \lambda \leq 1\}$$

若时刻 t 时, MT 的投影与无人机可达域 V_i 不相交, 则烟幕干扰弹不可能在 t 时刻的时候处于三维空间点 M , T 之间, 我们认为这样的情况无法达成对真实目标的有效遮蔽。

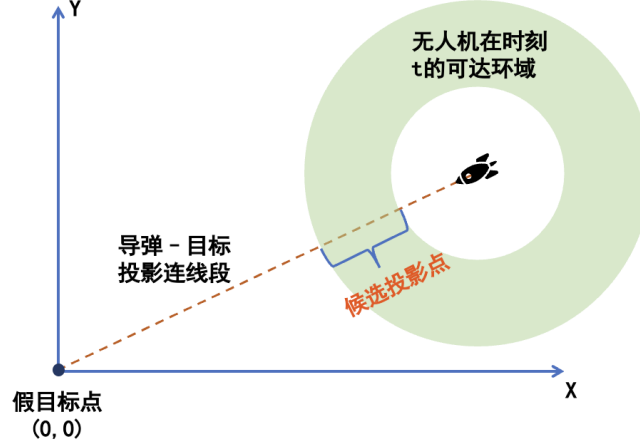


图9 无人机可行域可视化

(2) 烟幕干扰弹下落动力学约束

首先, 考虑导弹 $M1$ 与目标轴心 $T = (0, 200, 5)$ 的连线, 其在时刻 t 的投影线段为:

$$\bar{L}(t) = \{ \Pi(M_1(t)) + \lambda (\Pi(T) - \Pi(M_1(t))) \mid \lambda \in [0, 1] \}$$

其中 $\Pi(\cdot)$ 表示到 x - y 平面的投影。无人机 FYi 在时刻 t 的可达域为:

$$V_i(t) = \{(x, y) \mid v_{min}t \leq \sqrt{(x - x_i)^2 + (y - y_i)^2} \leq v_{max}t\}$$

若 $\bar{L}(t) \cap V_i(t) = \emptyset$, 则无人机不可能在时刻 t 将烟幕干扰弹送至导弹与目标之间, 从而无法形成有效遮蔽; 若交集非空, 则存在候选引爆点。

在三维空间中, 引入比例参数 $\lambda \in [0, 1]$, 候选点可表示为

$$P(\lambda, t) = (1 - \lambda) M_1(t) + \lambda T$$

即

$$P(\lambda, t) = ((1 - \lambda)x_m(t), (1 - \lambda)y_m(t) + 200\lambda, (1 - \lambda)z_m(t) + 5\lambda)$$

因此, 候选引爆点集合为

$$\mathcal{C}(t) = \{P(\lambda, t) \mid \lambda \in [0, 1], \Pi(P(\lambda, t)) \in V_i(t)\}$$

进一步考虑烟幕干扰弹在竖直方向的下落动力学: 设无人机初始高度为 Z_u , 候选点高度为 Z_c , 重力加速度为 g , 则自由落体时间满足

$$\Delta = \sqrt{\frac{2(Z_u - Z_c)}{g}}$$

若 $\Delta \leq t$, 则烟幕干扰弹可以在释放后自由下落到达该候选点。其释放时刻为:

$$t_{rel} = t - \Delta \quad (t_{rel} \geq 0)$$

(3) 离散化时间步长

在连续时间域上直接求解上述约束条件难度较高, 且我们的目标只是得到一个初步可行解。因此为便于数值实现, 我们引入时间离散化处理。

设导弹从被发现时刻 $t = 0$ 开始，最大飞行时间为 t_{max} ，我们在区间 $[0, t_{max}]$ 以固定步长 Δt 进行离散采样：

$$t_k = k \Delta t, \quad k = 0, 1, 2, \dots, \left\lfloor \frac{t_{max}}{\Delta t} \right\rfloor$$

对每一个离散时刻 t_k ，可得到对应的候选爆心点集合 $\mathcal{C}(t_k)$ ，并进一步筛选出满足动力学约束的可行点。最终，离散化候选点集合表示为：

$$\mathcal{C}_{ds} = \bigcup_k \mathcal{C}(t_k)$$

(4) 候选点离散化 λ 的处理

在每一个离散时间步 t_k 上，候选引爆点 $\mathcal{C}(t_k)$ 由参数 $\lambda \in [0, 1]$ 所确定。然而 λ 为连续变量，若直接搜索将导致计算量过大。为此，本文采用如下离散化策略：

1. 边界根计算。首先将投影线段 $\bar{L}(t_k)$ 与无人机可达域的内圆

$$r_{min} = v_{min} t_k, \quad r_{max} = v_{max} t_k$$

λ 根记为 $\{\lambda_b\}$ ，这些点代表线段恰好落在可达域边界上的交点。

2. 区间中点采样。将所有边界根按大小排序，记为 $0 = \lambda_0 < \lambda_1 < \dots < \lambda_m = 1$ 。在相邻根之间取中点：

$$\lambda_{j+\frac{1}{2}} = \frac{\lambda_j + \lambda_{j+1}}{2}, \quad j = 0, 1, \dots, m-1,$$

并判断其对应投影点是否位于圆环带 $[r_{min}, r_{max}]$ 则将该中点采样作为候选解。

3. 集合合并。最终在 t_k 时刻的离散候选 λ 集合 $\Lambda(t_k)$ 为：

$$\Lambda(t_k) = \{\lambda_b\} \cup \{\lambda_{j+\frac{1}{2}} \mid \Pi \left(P \left(\lambda_{j+\frac{1}{2}}, t_k \right) \right) \in V_i(t_k) \}$$

于是，时刻 t_k 的候选引爆点集合可以写为

$$\mathcal{C}(t_k) = \{ P(\lambda, t_k) \mid \lambda \in \Lambda(t_k) \}$$

5.4.2 候选引爆点的二次筛选模型的建立

(1) 张角判别法

经过上面的初步处理，我们可以得到约 240×6 组 (t, λ) 值，每组 (t, λ) 值都对应了一个可疑的候选起爆点。为优化算法效率，避免对每一个候选点都执行完整的遮蔽情况判定，我们会利用张角判别法对候选点进行初步筛选。

具体而言，我们比较“导弹视角下目标圆柱的张角”与“导弹视角下烟幕云球的张角”：

1. 目标张角 α

目标为半径 $R_{cyl} = 7\text{m}$ 、高度 $H_{cyl} = 10\text{m}$ 的直圆柱。从导弹位置 $\mathbf{M}(t)$ 出发，目标的“有效半径”取为半对角线：

$$r_{eff} = \sqrt{R_{cyl}^2 + \left(\frac{H_{cyl}}{2} \right)^2}$$

视距取导弹到圆柱轴心中点 $\mathbf{A} = (0, 200, 5)$ 的距离：

$$d_{target} = \| \mathbf{A} - \mathbf{M}(t) \|$$

于是目标张角近似为：

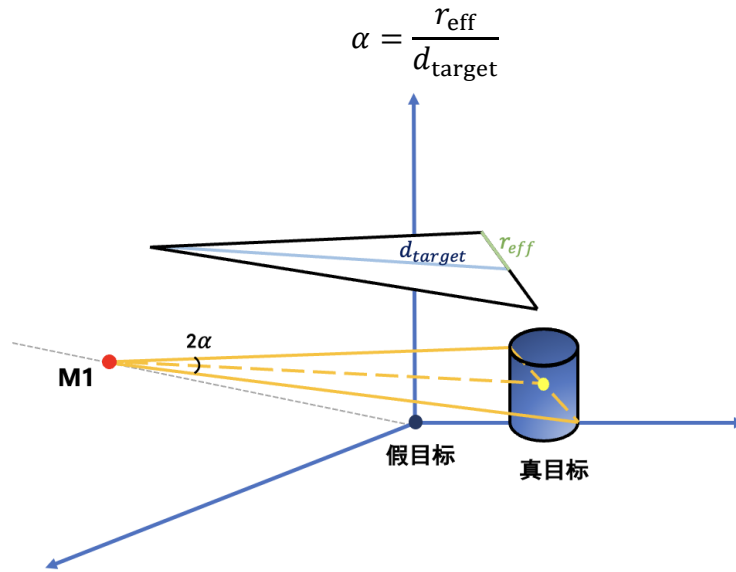


图10 目标张角示意图

2.云团张角 φ

烟幕云球半径 $R_{\text{cloud}} = 10\text{m}$ 。从导弹位置 $\mathbf{M}(t)$ 出发，云心为 $\mathbf{C}$,视距：

$$d_{\text{cloud}} = \|\mathbf{C} - \mathbf{M}(t)\|$$

对应的云团张角为：

$$\varphi = \frac{R_{\text{cloud}}}{d_{\text{cloud}}}$$

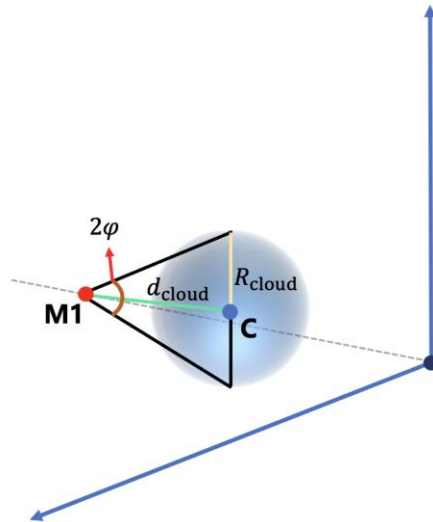


图11 云团张角示意图

3.判别准则

若 $\varphi \geq 0.8\alpha$,则认为该云团可能完全遮蔽目标圆柱，可以进入后续更精细的“线段-球体相交”验证。其中系数 0.8 为松弛因子，避免因近似而错杀潜在可行解。

该方法作为快速筛选，仅需一次比值运算即可大幅减少后续需要精细检测的候选点数量，从而显著提升算法效率。

(2) 基于遮蔽判决函数的遮蔽情况判定

基于问题一给出的遮蔽判断函数，我们进一步检测候选点在起爆后 20s 内，对导弹的遮蔽时长情况，并依遮蔽时长对候选解执行排序。对于每个无人机，我们都分别依据上述算法流程执行处理，保留排序结果的 top50。

(3) 候选解空间搜索

三架无人机，每架都提供了 50 种候选可行起爆点。我们在 $50 \times 50 \times 50$ 的空间内执行组合搜索，求得遮蔽时间段并集最长者，作为最优初步策略 $\pi^* = (t^*, \lambda^*)$ 。

5.4.3 多机单烟单导弹优化模型的建立与求解

(1) 基于 (t, λ) 反解决策变量

给定 (t, λ) ，则确定了空间中的引爆点 $P(\lambda, t)$ ，由于经过了前面的可行性分析，对于得到最优初步策略 $\pi^* = (t^*, \lambda^*)$ ，其对应的引爆点 $P(\lambda^*, t^*)$ ，必然是可行的。对于无人机 FY_i 我们可以根据起爆点坐标 $P(\lambda^*, t^*)$ 反解出决策变量 $d_i^* = (v_f, \theta, t_{rel}, t_e)$

引爆时间

$$t_e = t^*$$

航向角

$$\theta = \text{atan2}(P_y - F_y, P_x - F_x)$$

释放时刻

$$t_{rel} = t_e - \sqrt{\frac{2(z_0 - P_z)}{g}}$$

水平速度大小

$$v_f = \frac{\| \mathbf{P}_{xy} - \mathbf{F}_{xy} \|}{t_{rel}}$$

基于以上公式，针对每个无人机，我们都能反解出 $\pi^* = (t^*, \lambda^*)$ 所对应的决策变量 d_i^* 。

(2) 基于搜索解的粒子群初始化

根据搜索得到的最优初步策略 $\pi^* = (t^*, \lambda^*)$ ，反解得到三架无人机的决策变量

$$d_i^* = (v_{f,i}^*, \theta_i^*, t_{rel,i}^*, t_{e,i}^*), \quad i = 1, 2, 3$$

将它们拼接成种子解

$$\mathbf{x}^{(0)} = (d_1^*, d_2^*, d_3^*).$$

在粒子群初始化时：第 0 个粒子直接取 $\mathbf{x}^{(0)}$ ；其余 $N - 1$ 个粒子在 $\mathbf{x}^{(0)}$ 附近加入高斯扰动生成：

$$\mathbf{x}^{(i)} = \Pi_{\mathcal{X}}(\mathbf{x}^{(0)} + \boldsymbol{\eta}_i \odot \text{span}), \quad \boldsymbol{\eta}_i \sim \mathcal{N}(0, 0.03^2 I),$$

其中 span 为各维变量的取值范围， $\Pi_{\mathcal{X}}$ 为投影到可行域的算子。

(3) 目标函数

我们需要最大化三朵烟幕云团对目标的有效遮蔽总时长，因此目标函数定义为：

$$J(\mathbf{x}) = \sum_{k=0}^K \mathbf{1}\{\mathcal{O}(t_k; \mathbf{x})\} \Delta t$$

其中 $t_k = t_{\min} + k\Delta t$, $k = 0, 1, \dots, K$; Δt 为离散化步长; $\mathbf{1}\{\cdot\}$ 为指示函数, 判断某段时间内导弹是否被遮蔽。

(4) 决策变量

决策变量为三架无人机各自的速度, 航向角, 投弹时间, 引爆时间。

$$\mathbf{x} = [v_{f,1}, \theta_1, t_{\text{rel},1}, t_{e,1}, v_{f,2}, \theta_2, t_{\text{rel},2}, t_{e,2}, v_{f,3}, \theta_3, t_{\text{rel},3}, t_{e,3}]^T \in R^{12}$$

(5) 约束条件

无人机速度约束:

$$v_f \in [70, 140] \text{ m/s}$$

投弹-引爆时序约束:

$$t_e \geq t_{\text{rel}} \geq 0.$$

(6) 粒子群-模拟退火更新策略

在迭代过程中, 粒子群算法用于全局搜索。每个粒子的位置随速度更新:

$$\mathbf{v}_i \leftarrow w \mathbf{v}_i + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i), \quad \mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

其中 $\mathbf{p}_i$ 为个体历史最优, $\mathbf{g}$ 为全局最优。

在每代末, 对当前全局最优解 $\mathbf{g}$ 进行小幅随机扰动, 若新解更优则接受; 若略差, 则以概率

$$\exp\left(\frac{J(\mathbf{y}) - J(\mathbf{g})}{T}\right)$$

接受, 其中 T 为逐步下降的“温度”。

这种方式结合了 PSO 的全局探索和 SA 的随机跳跃, 可以在保证收敛速度的同时, 降低陷入局部最优的风险。

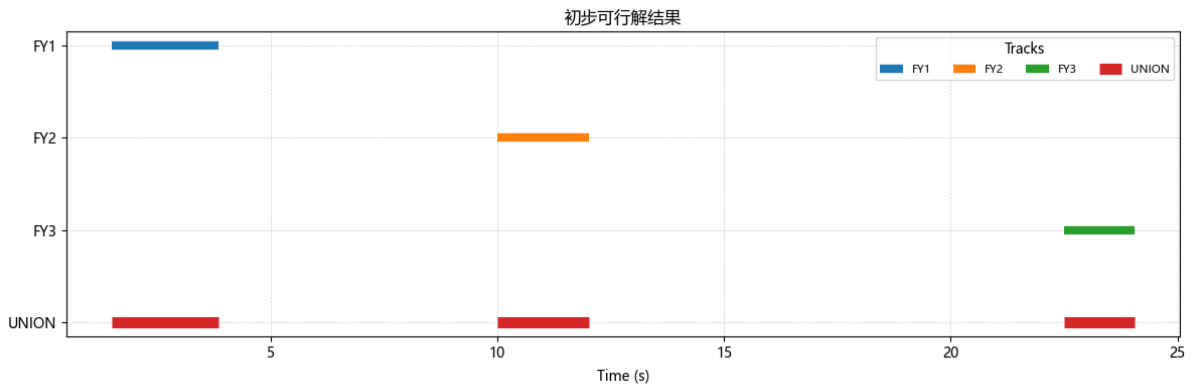


图12 初步筛选可行解可视化

上图展示经过初步筛选后所得可行解的可视化结果, 遮蔽情况为:

表5 初步筛选的可行解

无人机	速度 (m/s)	方向 (°)	投放时间 (s)	爆炸时间 (s)	干扰时长 (s)
FY1	104.91	5.95	0.60	1.50	2.34
FY2	140.00	286.03	4.29	10.00	2.02

无人机	速度 (m/s)	方向 (°)	投放时间 (s)	爆炸时间 (s)	干扰时长 (s)
FY3	138.19	89.14	18.15	22.50	1.55

总干扰时长为 5.91 s。

问题四求解结果

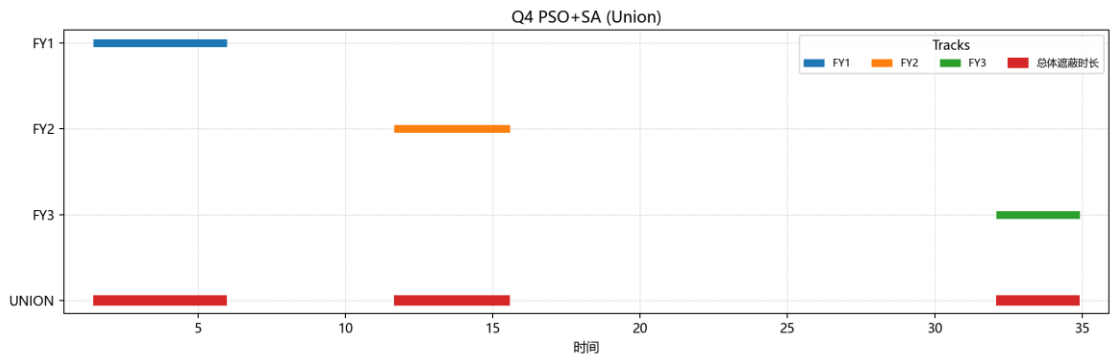


图13 基于初步解进一步优化得到的最终解

在初步可行解的基础上指导粒子群+模拟退火算法继续优化，最终得到一个更优的解。这个解在初步解的基础上更进一步执行优化，取得了更长的遮蔽时长，总遮蔽时长 11.560s，提升了 5.650s。

表6 进一步优化得到的最终解

无人机	速度 (m/s)	方向 (°)	投放时间 (s)	爆炸时间 (s)	干扰时长 (s)
FY1	105.85	6.07	0.69	0.70	4.56
FY2	140.00	286.16	4.47	10.05	3.95
FY3	136.58	89.08	18.51	22.73	3.05

如下图，即为该解对应的时间轴示意图。

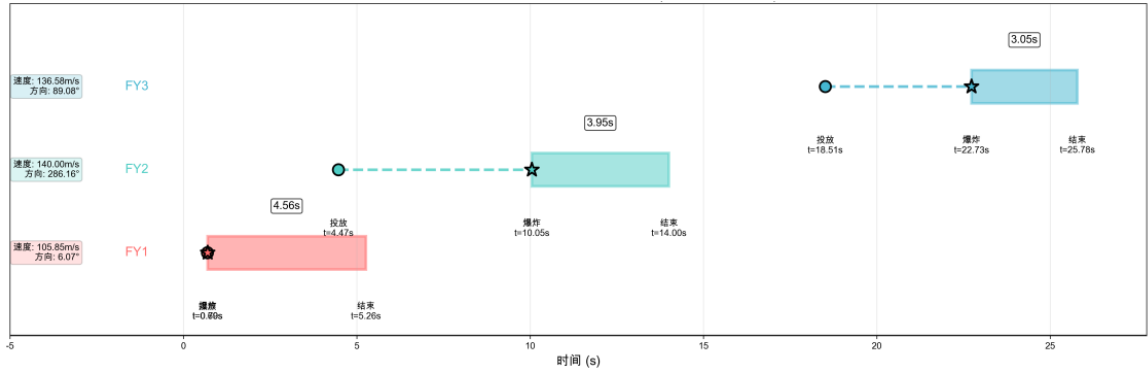


图14 问题三时间轴示意图

表7 问题四的求解结果

输出	FY1	FY2	FY3
无人机运动方向	5.62	288.35	88.59
无人机运动速度 (m/s)	140.00	140.00	140.00

烟幕干扰弹投放点的 x 坐标 (m)	17905.43	12209.73	6062.06
烟幕干扰弹投放点的 y 坐标 (m)	10.37	767.69	-474.87
烟幕干扰弹投放点的 z 坐标 (m)	1800.00	1400.00	700.00
烟幕干扰弹起爆点的 x 坐标 (m)	17910.08	12449.54	6076.31
烟幕干扰弹起爆点的 y 坐标 (m)	10.83	44.72	104.93
烟幕干扰弹起爆点的 z 坐标 (m)	1799.99	1254.95	615.91
有效干扰时长 (s)	4.58	3.96	3.06

5.5 问题五模型建立与求解

我们注意到，问题五是由前问层层铺垫而来的**复杂多目标优化问题**，其复杂性不仅在于多无人机、多烟幕干扰弹、多导弹的协同，还在于每架无人机都涉及到飞行角度、飞行速度、投弹时间、爆炸时间的调整，使得整体搜索空间呈指数级扩张。因此，我们从前问得到启发，对第四问的方法进行了迭代，继续改进了启发式候选算法，在**物理约束引导下缩小搜索空间**的基础上，嵌入**聚类降维与贪心算法**，获得了性能及其优异的**初始解**，同时**大大降低了计算开销**。并进一步使用粒子群优化算法，从而实现对这一复杂优化模型的高性能求解。

5.5.1 多机多烟多导弹干扰优化模型的建立

(1) 目标函数

本问模型与前文建模类似，区别只在无人机、烟幕干扰弹、导弹的数量与位置。由于导弹数目变多，前问有效遮蔽时长的定义在此问不再适用。

考虑实际意义，本题中，单个导弹的有效遮蔽时长定义为：**存在任意烟幕云团，使得该导弹被有效遮蔽的时间步的累加**。而总的有效遮蔽时长，就是**所有导弹的有效遮蔽时长之和**。所以，本问中有效遮蔽时长可以按如下步骤计算：

1. 将所有非空区间按起点排序。
2. 进行线性扫描合并重叠或相邻区间。
3. 合并后区间集合为 $\{\tilde{J}_\ell\}_{\ell=1}^L$ ，每个为 $[s_\ell, e_\ell]$ （互不重叠且按时间顺序）。
4. 则该导弹的遮蔽时长

$$T_j = \sum_{\ell=1}^L (e_\ell - s_\ell).$$

最终全局目标函数为 $F = \sum_{j=1}^M T_j$

(2) 决策变量

决策变量为五架无人机的飞行参数与投弹参数的组合，共 40 维（每架无人机含 2 个飞行参数、3 枚弹的 2 个投弹参数），具体形式为：

$$\mathbf{x} = \begin{bmatrix} v_1, \theta_1, t_{rel1,1}, t_{e1,1}, t_{rel1,2}, t_{e1,2}, t_{rel1,3}, t_{e1,3} \\ v_2, \theta_2, t_{rel2,1}, t_{e2,1}, t_{rel2,2}, t_{e2,2}, t_{rel2,3}, t_{e2,3} \\ \vdots \\ v_5, \theta_5, t_{rel5,1}, t_{e5,1}, t_{rel5,2}, t_{e5,2}, t_{rel5,3}, t_{e5,3} \end{bmatrix}$$

其中：

$v_i \in [70, 140]$ (无人机 i 的飞行速度, 单位 m/s);
 $\theta_i \in [0^\circ, 360^\circ]$ (无人机 i 的航向角, X 轴正方向为 0° , 逆时针为正);
 $t_{reli,k}$ (无人机 i 第 k 枚弹的投放时刻, 单位 s);
 $t_{ei,k}$ (无人机 i 第 k 枚弹的起爆时刻, 需满足 $t_{ei,k} \geq t_{reli,k} \geq 0$)。

(3) 约束条件

速度约束

$$70 \leq v_i \leq 140 (i = 1 - 5);$$

投弹间隔约束

对每架无人机 i 的第 k 、 $k+1$ 枚弹, $t_{reli,k+1} - t_{reli,k} \geq 1 (k = 1, 2)$;

时序约束

$$\text{对所有弹, } t_{ei,k} \geq t_{reli,k} \geq 0$$

烟幕有效性约束

烟幕起爆后 20s 内有效, 即 $t \in [t_{ei,k}, t_{ei,k} + 20]$ 时云团才参与遮蔽判定;

物理可行性约束

$$\text{烟幕弹下落时间 } \Delta t_{fall} = \sqrt{\frac{2(z_{ui} - z_{ei,k})}{g}} \leq t_{ei,k} - t_{reli,k}$$

(z_{ui} 为无人机初始高度, 确保弹能下落至起爆高度)

由于烟幕云团有效半径仅 10m, 而导弹初始位置与假目标距离达 18000~20000m, 云团尺度相对于整体空间尺度可忽略不计。因单架无人机的烟幕覆盖范围有限, 难以同时对两架及以上、飞行轨迹差异显著的导弹形成有效遮蔽, 此类作用对全局优化方向无决定性影响。据此, 可以暂不额外考虑同一无人机对多个导弹的干扰策略, 从而很大程度上简化问题。

5.5.2 改进的物理约束引导获取高性能初始解

(1) 候选引爆点的生成与筛选

首先需要基于物理约束, 生成一系列物理层面可行、遮蔽潜力较大的无人机-导弹导弹候选, 避免搜索空间过大, 带来大量无效计算。此步与第四问类似, 这里再次简单介绍步骤如下:

STEP1 对 M1-M3 三枚导弹, 分别按 0.5s 步长离散飞行时间, 起点为 0.5s, 终点为导弹抵达假目标的时间 $t_{j,max}$, 得到每枚导弹的起爆时间候选集合 $T_j = \{t_{j,1}, t_{j,2}, \dots\}$ 。

STEP2 烟幕云团落点解析计算对每架无人机 i 、导弹 j 、起爆时间 $t_e \in T_j$, 解析求解烟幕云团落点参数 $\lambda (\lambda \in [0, 1])$, 表示落点在导弹 t_e 时刻位置到目标中心连线上的比例):

- 计算无人机 i 在 t_e 时刻的可达域 (XY 平面内半径 $70t_e \sim 140t_e$ 的圆环);
- 计算 “导弹 t_e 时刻位置 → 目标中心” 连线的 XY 投影线段, 求该线段与无人机可达域的交集;
- 取交集的边界点与中点作为 λ 候选, 确保覆盖所有潜在有效落点, 得到 λ 列表。

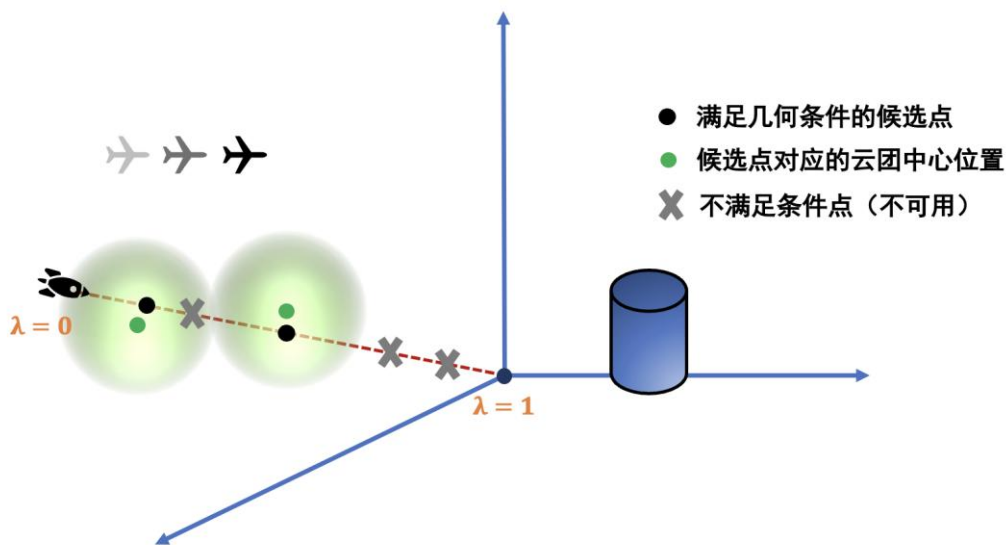


图15 候选引爆点筛选示意图

STEP3 快速筛选合适的候选引爆点。

张角门验证。首先计算导弹视角下“目标张角”（目标半对角线/导弹到目标距离）与“云团张角”（云团半径/导弹到云团距离），仅保留云团张角 $\geq 0.8 \times$ 目标张角的候选；

物理验证。反解无人机速度、航向、投放时刻，剔除速度超范围或下落时间不满足的候选；

单云遮蔽验证。计算该候选对导弹 j 的遮蔽时长，仅保留遮蔽时长 >0 的候选。

STEP4 候选排序与截取。对每对无人机-导弹对，按“单云遮蔽时长”降序排序，保留前 25 个候选，形成（无人机-导弹）候选字典，值为对应候选引爆点的列表。

(2) 航向-速度聚类降维与贪心选优

此步的目的是解决“单架无人机只能固定一个航向/速度”的约束，减少候选组合数量，同时利用贪心算法快速筛选性能较高的组合。

STEP1 对每架无人机 i ，合并其与 M1-M3 三枚导弹的所有候选（如 FY1 合并 FY1-M1、FY1-M2、FY1-M3 的候选），形成该无人机的全局候选列表。

STEP2 航向-速度聚类按方向-速度对候选聚类。

航向角：转为角度后按 1° 分桶（如 30.2° 与 30.8° 归为 30° 桶）；

飞行速度：按 1m/s 分桶（如 80.3m/s 与 80.9m/s 归为 80m/s 桶）；

相同的候选归为一个“簇”，簇内所有候选对应的无人机航向、速度视为一致。

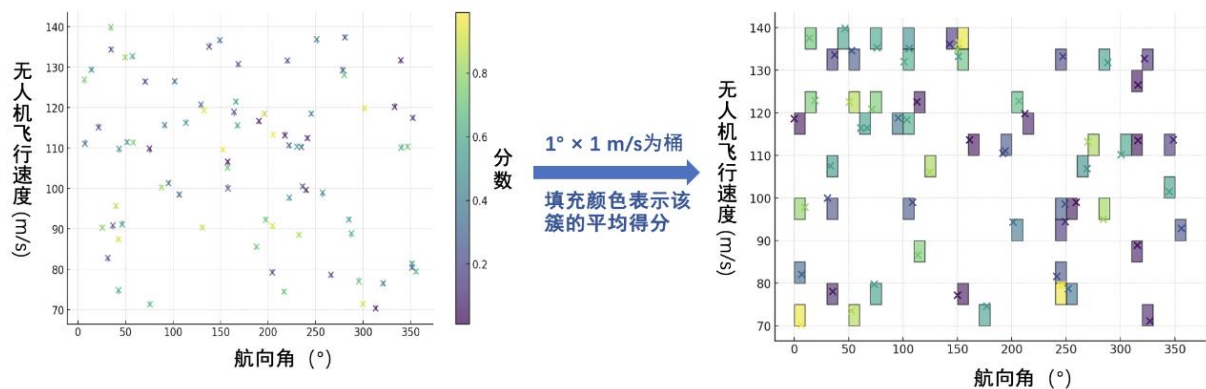


图16 聚类降维示意图

STEP3 簇内候选排序与裁剪每个簇内按有效遮蔽时长降序排序，剔除重复候选后保留前 10 个候选，减少后续组合计算量；同时检查簇内候选是否满足投弹间隔不少于 1s，剔除无效簇。

STEP4 生成无人机有效簇字典。每架无人机对应一个字典，键为（航向桶，速度桶），值为排序后的簇内候选列表，最终得到五架无人机的有效簇集合，为后续组合优化提供基础。

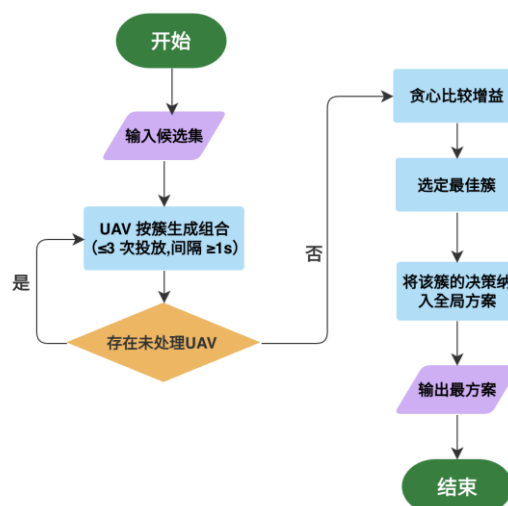


图17 贪心选优算法流程图

5.5.3 粒子群算法进一步优化得解

在前述步骤的基础上，我们获得了一个高性能的初始解。接下来即可沿用粒子群算法，扩大参数搜索范围。优化后，重新计算所有导弹的遮蔽并集时长，求和得到最终全局总遮蔽时长。

问题五求解结果

经过前文步骤中对解空间的预优化，所获得的高性能初始解如下图。在此解的情况下，总的有效遮蔽时长为 19.2s。

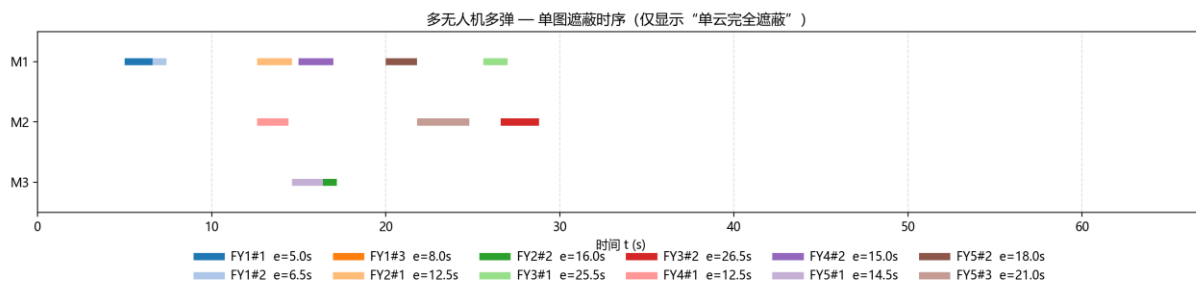


图18 初始解的遮蔽时序图

以该初始解为起点，运行粒子群优化算法。设定 $N = 200$, $K_{\max} = 100$, $c_1 = c_2 = 1.6$, $w = 1$, 求得该算法下的最优解如下。

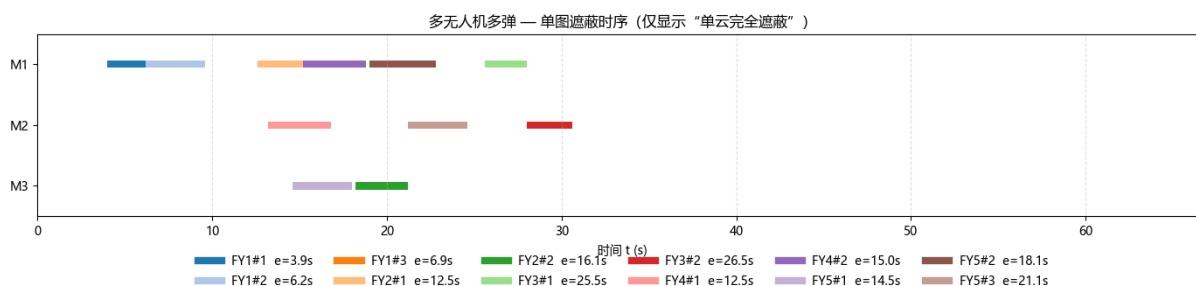


图19 最优解的遮蔽时序图

在此解的情况下，求得总的有效遮蔽时长为 **34.00s**。可以看出，智能优化算法在初始解的基础上进行了微调，进一步延长了各个烟幕弹的遮蔽时间，从而得到了更优解。详细参数已依照题目要求保存至 result3.xlsx。

六、模型的评价与改进

6.1 模型的优点

1. 物理建模完整，本文模型建立了涵盖导弹、无人机、干扰弹和烟幕云团的多层次动力学系统。
2. 模型采用“线段-球体相交”方法，建立了双锥遮蔽判定模型，来判定导弹-目标连线是否被云团阻断，将模型简化为只需判断圆柱体上下表面圆环是否成立，大大减少了模型复杂度，提高计算速率。
3. 对于低维参数空间问题，引入粒子群优化与遗传算法等智能优化方法，能够有效搜索全局最优。对于多无人机多导弹复杂对抗场景，采用物理约束引导的启发式候选生成、航向-速度聚类降维与贪心选优、粒子群全局优化多阶段算法，实现了在复杂决策空间中快速求解最优解。

6.2 模型的不足

1. 模型假设烟幕云团为球体并以固定速度下沉，未考虑大气环境因素，导致仿真结果与实际条件存在偏差。
2. 智能优化方法对迭代次数和种群规模等超参数较为敏感，启发式贪心方法则可能陷入局部最优。

6.3 模型的推广和改进

推广方向：

模型可推广至多个关键目标（如舰艇编队、机场设施）的协同防护任务，研究无人机群如何形成多片烟幕屏障以实现区域防御。可扩展至多波次、多类型导弹交替攻击的对抗问题，从而模拟更复杂、更贴近实战的作战环境。

改进方向：

1. 可以在现有球体下沉模型基础上，引入风场扰动、扩散过程和气象条件影响，提高模拟的可信度。
2. 在粒子群和遗传算法的基础上，引入动态参数调整、分布式并行进化或混合智能优化方法，减少对人工调参的依赖。
3. 用解析公式或快速判定方法替代部分时间步长扫描计算，提升大规模问题下的计算效率，以满足实时作战需求。

参考文献

- [1] 张良均. Python 数据分析与挖掘实战[M]. 机械工业出版社, 2016.
- [2] 茆诗松, 程依明, 濮晓龙, 等. 概率论与数理统计教程第二版[M]. 北京: 高等教育出版社, 2011
- [3] 《运筹学》教材编写组. 运筹学.第 4 版[M]. 清华大学出版社, 2012.
- [4] Rachel Schutt, Cathy O'Neil. 数据科学实战[M]. 人民邮电出版社, 2015.
- [5] 姜启源, 谢金星, 叶俊. 数学模型.第 4 版[M]. 高等教育出版社, 2011.
- [6] 韩中庚. 数学建模方法及其应用-第 2 版[M]. 高等教育出版社, 2009.
- [7] GPT-5, GPT-5-Reasoning-Alpha-20250713, OpenAI, 2025-09-07.
- [8] Claude-4-Sonnet, Claude-4-Sonnet-2025Q2-Update, Anthropic, 2025-09-07.
- [9] 豆包, Doubao-Knowledge-Enhanced-V2025.08, 字节跳动, 2025-09-07.
- [10] Kimi, Kimi-Advanced-2025.09.01, 北京月之暗面科技有限公司, 2025-09-07.

附录

附录 A: 支撑材料列表

支撑材料列表

序号	文件名	材料说明
1	problem_1.py	建立了无人机投放烟幕干扰弹的仿真模型，用于计算导弹飞行过程中目标被有效遮蔽的时长。
2	problem_2.py	实现了基于粒子群优化的算法，用于自动搜索无人机投放烟幕干扰弹的最优参数组合，最大化对导弹的有效遮蔽时长。
3	problem_2_genetic.py	实现了基于遗传算法的优化方法，用于搜索无人机投放烟幕干扰弹的最优参数组合，最大化导弹的有效遮蔽时长。
4	problem_3.py	基于三维无人机-烟幕干扰模型，使用粒子群优化算法求解三枚干扰弹投放与起爆时刻，使导弹完全遮蔽总时长最大。
5	problem_4.py	用于多无人机烟幕干扰优化问题，先验证给定投放方案的遮蔽效果，再定义用于 PSO+模拟退火优化的适应度函数和辅助函数，实现干扰时长评估、投放点计算及最终结果保存。
6	problem_4_equation.py	实现了多无人机、多导弹烟幕干扰启发式优化求解器，用于生成各无人机的最佳投放方案并计算总遮蔽时长，同时支持导出 Excel 启发点。
7	problem_5.py	实现了五架无人机对三枚导弹的启发式烟幕干扰优化求解，通过解析候选→聚类→簇内组合→贪心选择生成最佳投放方案并导出 Excel 结果。
8	p5ada.py	以生成的投放方案为初始解，进一步利用粒子群算法优化。
9	p5val.py	第五问可视化验证器
10	result1.xlsx	问题三的结果
11	result2.xlsx	问题四的结果
12	result3.xlsx	问题五的结果
13	AI 工具使用详情.pdf	介绍 AI 工具使用情况

附录 B: 题目解决问题所用代码

代码文件 1 问题一的求解 (problem_1.py)

```

import numpy as np
import time

class CInterferenceModel:
    def __init__(self):

        self.g = 9.8
        self.p_v = 300
        self.c_v = 3
        self.eff_r = 10
        self.eff_t = 20

        self.true_r = 7
        self.true_h = 10
        self.true_c = np.array([0, 200, 0])
        self.fake_s = np.array([0, 0, 0])

        self.M1_s = np.array([20000, 0, 2000])
        self.FY1_s = np.array([17800, 0, 1800])

        self.f_theta = 180.0

    def calculate_m_s(self, t):
        direction = (self.fake_s - self.M1_s) / np.linalg.norm(self.fake_s
- self.M1_s)
        return self.M1_s + direction * self.p_v * t

    def calculate_f_s(self, t, f_v=120):

        angle_rad = np.radians(self.f_theta)
        cos_angle = np.cos(angle_rad)
        sin_angle = np.sin(angle_rad)

        direction_xy = np.array([cos_angle, sin_angle])

        move_xy = direction_xy * f_v * t

        pos = np.array([
            self.FY1_s[0] + move_xy[0],
            self.FY1_s[1] + move_xy[1],
            self.FY1_s[2]
        ])
        return pos

    def calculate_p_s(self, release_time, f_v=120):
        release_pos = self.calculate_f_s(release_time, f_v)
        angle_rad = np.radians(self.f_theta)
        cos_angle = np.cos(angle_rad)
        sin_angle = np.sin(angle_rad)

        direction_xy = np.array([cos_angle, sin_angle])
        f_v_xy = direction_xy * f_v
        def c_s(t):
            if t < release_time:
                return None
            dt = t - release_time
            pos = release_pos.copy()
            pos[0] += f_v_xy[0] * dt
            pos[1] += f_v_xy[1] * dt
            pos[2] -= 0.5 * self.g * dt**2

```

```

        return pos

    return c_s

def calculate_c_center(self, explode_time, release_time, f_speed=120):
    c_s = self.calculate_p_s(release_time, f_speed)
    explode_pos = c_s(explode_time)

    def c_center(t):
        if t < explode_time:
            return None
        dt = t - explode_time

        pos = explode_pos.copy()
        pos[2] -= self.c_v * dt
        return pos

    return c_center

def get_true_points(self, num_points=36):
    angles = np.linspace(0, 2*np.pi, num_points, endpoint=False)

    top_points = []
    bottom_points = []

    for angle in angles:
        x = self.true_c[0] + self.true_r * np.cos(angle)
        y = self.true_c[1] + self.true_r * np.sin(angle)

        bottom_points.append([x, y, self.true_c[2]])
        top_points.append([x, y, self.true_c[2] + self.true_h])

    return np.array(top_points + bottom_points)

def line_sphere_intersection(self, line_start, line_end, sphere_center, sphere_radius):
    d = line_end - line_start
    f = line_start - sphere_center

    a = np.dot(d, d)
    b = 2 * np.dot(f, d)
    c = np.dot(f, f) - sphere_radius**2

    discriminant = b**2 - 4*a*c

    if discriminant < 0:
        return False

    sqrt_discriminant = np.sqrt(discriminant)
    t1 = (-b - sqrt_discriminant) / (2*a)
    t2 = (-b + sqrt_discriminant) / (2*a)

    return (0 <= t1 <= 1) or (0 <= t2 <= 1) or (t1 < 0 and t2 > 1)

def check_effective_hiding(self, m_s, c_center, true_points):
    if c_center is None:
        return False

    for point in true_points:
        if not self.line_sphere_intersection(point, m_s, c_center,

```

```

self.eff_r):
    return False

    return True

def set_f_angle(self, angle_deg):
    self.f_theta = angle_deg % 360.0
    print(f"无人机飞行角度已设置为: {self.f_theta:.1f}°")

def set_f_s(self, s):
    """设置无人机初始位置"""
    self.FY1_s = np.array(s)
    print(f"无人机初始位置已设置为: {self.FY1_s}")

def set_m_s(self, s):
    self.M1_s = np.array(s)
    print(f"导弹初始位置已设置为: {self.M1_s}")

def set_true_center(self, s):
    self.true_c = np.array(s)
    print(f"真目标中心已设置为: {self.true_c}")

def set_fake_s(self, s):
    self.fake_s = np.array(s)
    print(f"假目标位置已设置为: {self.fake_s}")

def reset_to_problem1(self):
    self.FY1_s = np.array([17800, 0, 1800])
    self.M1_s = np.array([20000, 0, 2000])
    self.f_theta = 180.0 # 180 度, 沿 x 轴负方向
    self.true_c = np.array([0, 200, 0])
    self.fake_s = np.array([0, 0, 0])
    print("已重置为问题 1 的默认参数")

def print_current_config(self):
    print("=== 当前配置 ===")
    print(f"无人机初始位置: {self.FY1_s}")
    print(f"导弹初始位置: {self.M1_s}")
    print(f"无人机飞行角度: {self.f_theta:.1f}°")
    print(f"真目标中心: {self.true_c}")
    print(f"假目标位置: {self.fake_s}")
    f_direction = self.get_f_direction()
    print(f"无人机飞行方向: [{f_direction[0]:.3f}, {f_direc-
tion[1]:.3f}])")
    print("=====")

def get_f_direction(self):
    angle_rad = np.radians(self.f_theta)
    cos_angle = np.cos(angle_rad)
    sin_angle = np.sin(angle_rad)

    direction_xy = np.array([cos_angle, sin_angle])

    return direction_xy

def solve_c_interference(self,
                          f_init_pos=None,
                          m_init_pos=None,

```

```

        f_angle=None,
        f_v=120,
        release_time=1.5,
        explode_delay=3.6,
        true_center=None,
        fake_s=None,
        verbose=True):

    start_time = time.time()

    original_f_pos = self.FY1_s.copy()
    original_m_pos = self.M1_s.copy()
    original_angle = self.f_theta
    original_true_center = self.true_c.copy()
    original_fake_s = self.fake_s.copy()

    # 设置参数
    if f_init_pos is not None:
        self.FY1_s = np.array(f_init_pos)
    if m_init_pos is not None:
        self.M1_s = np.array(m_init_pos)
    if f_angle is not None:
        self.f_theta = f_angle % 360.0 # 规范化到 [0, 360]度
    if true_center is not None:
        self.true_c = np.array(true_center)
    if fake_s is not None:
        self.fake_s = np.array(fake_s)

    explode_time = release_time + explode_delay

    if verbose:
        print("=== 烟雾干扰求解 ===")
        print("参数设置: ")
        print(f"无人机初始位置: {self.FY1_s}")
        print(f"导弹初始位置: {self.M1_s}")
        print(f"无人机速度: {f_v} m/s")
        print(f"无人机飞行角度: {self.f_theta:.1f}°")
        flight_direction = self.get_f_direction()
        print(f"无人机飞行方向: [{flight_direction[0]:.3f}, {flight_direc-
tion[1]:.3f}])")
        print(f"真目标中心: {self.true_c}")
        print(f"假目标位置: {self.fake_s}")
        print(f"投放时间: {release_time} s")
        print(f"起爆延时: {explode_delay} s")
        print(f"起爆时间: {explode_time} s")
        print()

    true_points = self.get_true_points()

    release_pos = self.calculate_f_s(release_time, f_v)
    c_trajectory = self.calculate_p_s(release_time, f_v)
    explode_pos = c_trajectory(explode_time)
    c_center_func = self.calculate_c_center(explode_time, release_time,
f_v)

    if verbose:
        print(f"目标采样点总数: {len(true_points)}")
        print(f"烟雾弹投放位置: {release_pos}")

```

```

        print(f"烟雾弹起爆位置: {explode_pos}")
        print()

        effective_periods = []
        dt = 0.01
        max_time = explode_time + self.eff_t + 5
        current_start = None

        for t in np.arange(explode_time, max_time, dt):
            m_pos = self.calculate_m_s(t)
            c_center = c_center_func(t)

            if t > explode_time + self.eff_t:
                if current_start is not None:
                    effective_periods.append((current_start, explode_time +
self.eff_t))
                break

            is_effective = self.check_effective_hiding(m_pos, c_center,
true_points)

            if is_effective and current_start is None:
                current_start = t
            elif not is_effective and current_start is not None:
                effective_periods.append((current_start, t))
                current_start = None

            if current_start is not None:
                effective_periods.append((current_start, min(max_time, ex-
plode_time + self.eff_t)))

        total_shielding_time = sum(end - start for start, end in effec-
tive_periods)

        end_time = time.time()
        runtime = end_time - start_time

        result_info = {
            'drone_init_pos': self.FY1_s.copy(),
            'missile_init_pos': self.M1_s.copy(),
            'drone_flight_angle': self.f_theta,
            'drone_speed': f_v,
            'release_time': release_time,
            'explode_delay': explode_delay,
            'explode_time': explode_time,
            'release_pos': release_pos,
            'explode_pos': explode_pos,
            'target_center': self.true_c.copy(),
            'fake_target': self.fake_s.copy(),
            'flight_direction': self.get_f_direction(),
            'runtime': runtime
        }

        if verbose:
            print("=== 计算结果 ===")
            print(f"有效遮蔽时间段: {effective_periods}")
            print(f"总有效遮蔽时长: {total_shielding_time:.3f} 秒")
            print(f"程序运行时长: {runtime:.3f} 秒")

```

```

        self.FY1_s = original_f_pos
        self.M1_s = original_m_pos
        self.f_theta = original_angle
        self.true_c = original_true_center
        self.fake_s = original_fake_s

    return total_shielding_time, effective_periods, result_info

def solve_problem1(self):
    start_time = time.time()

    print("=== 问题1 求解 ===")
    print("参数设置: ")
    print(f"无人机 FY1 初始位置: {self.FY1_s}")
    print(f"导弹 M1 初始位置: {self.M1_s}")
    print(f"无人机速度: 120 m/s")
    print(f"无人机飞行角度: {self.f_theta:.1f}°")
    f_direction = self.get_f_direction()
    print(f"无人机飞行方向: [{f_direction[0]:.3f}, {f_direc-
tion[1]:.3f}]")
    print(f"投放延时: 1.5 s")
    print(f"起爆延时: 3.6 s")
    print()

    release_time = 1.5
    explode_delay = 3.6
    explode_time = release_time + explode_delay
    f_speed = 120

    target_points = self.get_true_points()
    print(f"目标采样点总数: {len(target_points)}")

    release_pos = self.calculate_f_s(release_time, f_speed)
    c_trajectory = self.calculate_p_s(release_time, f_speed)
    explode_pos = c_trajectory(explode_time)
    c_center_func = self.calculate_c_center(explode_time, release_time,
f_speed)

    print(f"烟雾弹投放位置: {release_pos}")
    print(f"烟雾弹起爆位置: {explode_pos}")
    print()

    effective_periods = []
    dt = 0.01
    max_time = explode_time + self.eff_t + 5

    current_start = None

    for t in np.arange(explode_time, max_time, dt):
        m_pos = self.calculate_m_s(t)
        c_center = c_center_func(t)

        if t > explode_time + self.eff_t:
            if current_start is not None:
                effective_periods.append((current_start, explode_time +
self.eff_t))
            break

```

```

        is_effective = self.check_effective_hiding(m_pos, c_center,
target_points)

        if is_effective and current_start is None:
            current_start = t
        elif not is_effective and current_start is not None:
            effective_periods.append((current_start, t))
            current_start = None

        if current_start is not None:
            effective_periods.append((current_start, min(max_time, ex-
plode_time + self.eff_t)))

        total_shielding_time = sum(end - start for start, end in effec-
tive_periods)

        end_time = time.time()
        runtime = end_time - start_time

        print("=== 计算结果 ===")
        print(f"有效遮蔽时间段: {effective_periods}")
        print(f"总有效遮蔽时长: {total_shielding_time:.3f} 秒")
        print(f"程序运行时长: {runtime:.3f} 秒")

        return total_shielding_time, effective_periods

if __name__ == "__main__":
    model = CInterferenceModel()
    shielding_time, periods = model.solve_problem1()

```

代码文件 2 问题二使用粒子群优化算法求解 (problem_2.py)

```

import numpy as np
import matplotlib.pyplot as plt
import time
from problem_1 import CInterferenceModel

class OptimizedCInterferenceModel(CInterferenceModel):
    def __init__(self):
        super().__init__()
        self.min_f_speed = 70.0
        self.max_f_speed = 140.0

    def calculate_occlusion_duration(self, f_angle, f_speed, release_time,
explode_delay):
        if not (self.min_f_speed <= f_speed <= self.max_f_speed):
            return 0.0
        if release_time < 0.0 or explode_delay < 0.0:
            return 0.0

        try:
            total_shielding_time, effective_periods, result_info =
self.solve_c_interference(

```

```

        f_angle=f_angle,
        f_speed=f_speed,
        release_time=release_time,
        explode_delay=explode_delay,
        verbose=False
    )
    return total_shielding_time
except:
    return 0.0

def pso_optimize(self, n_parcs=50, n_iters=35, random_seed=1, verbose=True):
    start_time = time.time()
    if verbose:
        print("=== 问题 2: PSO 优化求解 ===")
        print(f"粒子数量: {n_parcs}")
        print(f"迭代次数: {n_iters}")
        print(f"随机种子: {random_seed}")
        print()

    rng = np.random.default_rng(random_seed)
    bounds = np.array([
        [0.0, 360.0],
        [self.min_f_speed, self.max_f_speed],
        [0.0, 70.0],
        [0.0, 20.0],
    ], dtype=float)

    dim = bounds.shape[0]

    pos = bounds[:, 0] + (bounds[:, 1] - bounds[:, 0]) * rng.random((n_parcs, dim))
    vel = np.zeros_like(pos)

    def fitness(params):
        f_angle, f_speed, release_time, explode_delay = params
        return self.calculate_occlusion_duration(f_angle, f_speed,
                                                release_time, explode_delay)

    fit = np.array([fitness(p) for p in pos])
    p_best = pos.copy()
    p_best_fit = fit.copy()

    gi = np.argmax(fit)
    g_best = pos[gi].copy()
    g_best_fit = fit[gi]

    w_start, w_end = 0.8, 0.3
    c1, c2 = 1.49, 1.49
    if verbose:
        print(f"初始最优适应度: {g_best_fit:.3f}")
        print(f"动态惯性权重: {w_start} → {w_end}")

    for it in range(n_iters):
        w = w_start - (w_start - w_end) * it / (n_iters - 1) if n_iters > 1 else w_start

        r1 = rng.random((n_parcs, dim))
        r2 = rng.random((n_parcs, dim))

```

```

pos)
    vel = w * vel + c1 * r1 * (p_best - pos) + c2 * r2 * (g_best -
pos)
    pos = pos + vel

    for d in range(dim):
        low, high = bounds[d]
        mask_low = pos[:, d] < low
        mask_high = pos[:, d] > high
        vel[mask_low, d] *= -0.5
        vel[mask_high, d] *= -0.5
        pos[:, d] = np.clip(pos[:, d], low, high)

    fit = np.array([fitness(p) for p in pos])

    improved = fit > p_best_fit
    p_best[improved] = pos[improved]
    p_best_fit[improved] = fit[improved]

    gi = np.argmax(fit)
    if fit[gi] > g_best_fit:
        g_best = pos[gi].copy()
        g_best_fit = fit[gi]

    if verbose and (it + 1) % 10 == 0:
        print(f"迭代 {it + 1}: 最优适应度 = {g_best_fit:.3f}, 当前惯性权
重 w = {w:.3f}")

    end_time = time.time()
    runtime = end_time - start_time

    if verbose:
        print(f"\n 优化完成, 用时: {runtime:.3f} 秒")

    return g_best, g_best_fit

def solve_problem2(self, n_par=200, n_iters=400, random_seed=1,
plot_results=True):
    best_params, best_time = self.pso_optimize(n_par, n_iters, ran-
dom_seed)

    f_angle_opt, f_speed_opt, release_time_opt, explode_delay_opt =
best_params
    explode_time_opt = release_time_opt + explode_delay_opt

    print("\n=== PSO 最优解 (问题 2) ===")
    print(f"无人机飞行角度  $\theta$  (度): {f_angle_opt:.2f}°")
    print(f"无人机飞行速度 v (m/s): {f_speed_opt:.3f}")
    print(f"烟雾弹投放时刻 t_rel (s): {release_time_opt:.3f}")
    print(f"起爆延时  $\tau_{burst}$  (s): {explode_delay_opt:.3f}")
    print(f"起爆时刻 t_burst (s): {explode_time_opt:.3f}")
    print(f"最大有效遮蔽时长 (s): {best_time:.3f}")

    total_shielding_time, effective_periods, result_info =
self.solve_c_interference(
        f_angle=f_angle_opt,
        f_speed=f_speed_opt,
        release_time=release_time_opt,

```

```

        explode_delay=explode_delay_opt,
        verbose=True
    )

    if plot_results:
        self.plot_occlusion_timeline(result_info)

    return best_params, best_time, result_info

def plot_occlusion_timeline(self, result_info):
    explode_time = result_info['explode_time']

    dt = 0.05
    ts = np.arange(explode_time, explode_time + self.effective_time +
1e-9, dt)
    occl = []

    original_angle = self.f_angle
    self.f_angle = result_info['f_angle']

    try:
        true_points = self.get_true_points()

        c_center_func = self.calculate_c_center(
            explode_time, result_info['release_time'], re-
sult_info['f_speed']
        )

        for t in ts:
            m_pos = self.calculate_m_trajectory(t)
            c_center = c_center_func(t)

            if c_center is None or t > explode_time + self.effec-
tive_time:
                occl.append(0.0)
            else:
                is_effective = self.check_effective_shielding(m_pos,
c_center, true_points)
                occl.append(1.0 if is_effective else 0.0)

        finally:
            self.f_angle = original_angle

    plt.figure(figsize=(10, 4))
    plt.plot(ts, occl, linewidth=2, color='blue')
    plt.xlabel("Time t (s)")
    plt.ylabel("Occlusion Status (1=Fully Occluded)")
    plt.title("Complete Occlusion Timeline (Optimal Solution)")
    plt.grid(True, alpha=0.3)
    plt.ylim(-0.1, 1.1)
    plt.show()

if __name__ == "__main__":
    model = OptimizedCInterferenceModel()

    # 求解问题2
    best_params, best_time, result_info = model.solve_problem2(
        n_parc=50,
        n_iters=80,

```

```

        random_seed=1,
        plot_results=True
    )

```

代码文件 3 问题二使用遗传算法优化求解(problem_2_genetic.py)

```

import numpy as np
import matplotlib.pyplot as plt
import time
from multiprocessing import cpu_count
from concurrent.futures import ProcessPoolExecutor
from problem_1 import CInterferenceModel

def calculate_fitness_parallel(args):
    individual, model_params = args

    temp_model = GeneticCInterferenceModel()

    for key, value in model_params.items():
        setattr(temp_model, key, value)

    f_angle, f_speed, release_time, explode_delay = individual
    return temp_model.calculate_occlusion_duration(
        f_angle, f_speed, release_time, explode_delay
    )

class GeneticCInterferenceModel(CInterferenceModel):
    def __init__(self):
        super().__init__()
        self.min_f_speed = 70.0
        self.max_f_speed = 140.0

    def calculate_occlusion_duration(self, f_angle, f_speed, release_time,
    explode_delay):
        if not (self.min_f_speed <= f_speed <= self.max_f_speed):
            return 0.0
        if release_time < 0.0 or explode_delay < 0.0:
            return 0.0

        try:
            total_shielding_time, effective_periods, result_info =
self.solve_c_interference(
                f_angle=f_angle,
                f_speed=f_speed,
                release_time=release_time,
                explode_delay=explode_delay,
                verbose=False
            )
            return total_shielding_time
        except:
            return 0.0

    def genetic_optimize(self, pop_size=100, n_generations=200, muta-
    tion_rate=0.15,
                                crossover_rate=0.8, elite_ratio=0.1, random_seed=1,
    verbose=True,

```

```

        use_parallel=True, n_processes=None):

    start_time = time.time()

    if n_processes is None:
        n_processes = cpu_count()

    n_processes = min(n_processes, pop_size)

    if verbose:
        print("=== 问题 2: 遗传算法优化求解 ===")
        print(f"种群大小: {pop_size}")
        print(f"进化代数: {n_generations}")
        print(f"变异率: {mutation_rate}")
        print(f"交叉率: {crossover_rate}")
        print(f"精英保留比例: {elite_ratio}")
        print(f"随机种子: {random_seed}")
        print(f"并行计算: {'是' if use_parallel else '否'}")
        if use_parallel:
            print(f"并行进程数: {n_processes}")
        print()

    rng = np.random.default_rng(random_seed)

    bounds = np.array([
        [0.0, 360.0],
        [self.min_f_speed, self.max_f_speed],
        [0.0, 50.0],
        [0.0, 10.0],
    ], dtype=float)
    dim = bounds.shape[0]
    n_elite = max(1, int(pop_size * elite_ratio))

    population = []

    n_random = int(pop_size * 0.7)
    random_pop = bounds[:, 0] + (bounds[:, 1] - bounds[:, 0]) *
rng.random((n_random, dim))
    population.extend(random_pop)

    n_guided = pop_size - n_random
    for _ in range(n_guided):
        angle = rng.normal(180, 45) % 360
        speed = rng.normal(105, 20)
        speed = np.clip(speed, self.min_f_speed, self.max_f_speed)
        release_time = rng.exponential(10)
        release_time = np.clip(release_time, 0, 50)

        explode_delay = rng.normal(5, 2)
        explode_delay = np.clip(explode_delay, 0, 10)

        population.append([angle, speed, release_time, explode_delay])

    population = np.array(population)

    def fitness(params):
        f_angle, f_speed, release_time, explode_delay = params
        return self.calculate_occlusion_duration(f_angle, f_speed,
                                                release_time, explode_delay)

```

```

def calculate_fitness_batch(population_batch):
    if use_parallel and len(population_batch) > 1:
        model_params = {
            'min_f_speed': self.min_f_speed,
            'max_f_speed': self.max_f_speed,
            'm_speed': self.m_speed,
            'effective_time': self.effective_time,
            'effective_radius': self.effective_radius,
            'Ml_init': self.Ml_init.copy(),
            'FYl_init': self.FYl_init.copy(),
            'true_center': self.true_center.copy(),
            'fake_target': self.fake_target.copy(),
            'f_angle': self.f_angle
        }

        args_list = [(individual, model_params) for individual in
population_batch]

        with ProcessPoolExecutor(max_workers=n_processes) as execu-
tor:
            fitness_values = list(executor.map(calculate_fitness_par-
allel, args_list))

        return np.array(fitness_values)
    else:
        return np.array([fitness(individual) for individual in popu-
lation_batch])

fitness_values = calculate_fitness_batch(population)

best_idx = np.argmax(fitness_values)
best_params = population[best_idx].copy()
best_fitness = fitness_values[best_idx]

if verbose:
    print(f"初始最优适应度: {best_fitness:.3f}")
    print(f"初始种群中有效解数量: {np.sum(fitness_values > 0)}")

initial_mutation_rate = mutation_rate
stagnation_counter = 0
last_best_fitness = best_fitness

for generation in range(n_generations):
    if generation > 0 and best_fitness == last_best_fitness:
        stagnation_counter += 1
        if stagnation_counter > 20:
            mutation_rate = min(0.3, initial_mutation_rate * 1.5)
        else:
            mutation_rate = initial_mutation_rate
    else:
        stagnation_counter = 0
        mutation_rate = initial_mutation_rate

    last_best_fitness = best_fitness

    new_population = []

    elite_indices = np.argsort(fitness_values)[-n_elite:]
    for idx in elite_indices:

```

```

        new_population.append(population[idx].copy())

    while len(new_population) < pop_size:
        parent1 = self.tournament_selection(population, fitness_val-
ues, rng, tournament_size=5)
        parent2 = self.tournament_selection(population, fitness_val-
ues, rng, tournament_size=5)

        if rng.random() < crossover_rate:
            child1, child2 = self.crossover(parent1, parent2, rng)
        else:
            child1, child2 = parent1.copy(), parent2.copy()

        child1 = self.mutate(child1, bounds, mutation_rate, rng)
        child2 = self.mutate(child2, bounds, mutation_rate, rng)

        new_population.extend([child1, child2])

    population = np.array(new_population[:pop_size])

    fitness_values = calculate_fitness_batch(population)

    current_best_idx = np.argmax(fitness_values)
    if fitness_values[current_best_idx] > best_fitness:
        best_params = population[current_best_idx].copy()
        best_fitness = fitness_values[current_best_idx]

    if verbose and (generation + 1) % 25 == 0:
        avg_fitness = np.mean(fitness_values)
        valid_solutions = np.sum(fitness_values > 0)
        print(f"第 {generation + 1} 代: 最优适应度 = {best_fit-
ness:.3f}, 平均适应度 = {avg_fitness:.3f}, 有效解数量 = {valid_solutions}, 变
异率 = {mutation_rate:.3f}")

    end_time = time.time()
    runtime = end_time - start_time

    if verbose:
        print(f"\n 优化完成, 用时: {runtime:.3f} 秒")

    return best_params, best_fitness

def tournament_selection(self, population, fitness_values, rng, tour-
nament_size=3):
    tournament_indices = rng.choice(len(population), tournament_size,
replace=False)
    tournament_fitness = fitness_values[tournament_indices]
    winner_idx = tournament_indices[np.argmax(tournament_fitness)]
    return population[winner_idx].copy()

def crossover(self, parent1, parent2, rng):
    alpha = rng.random()
    child1 = alpha * parent1 + (1 - alpha) * parent2
    child2 = (1 - alpha) * parent1 + alpha * parent2
    return child1, child2

def mutate(self, individual, bounds, mutation_rate, rng):
    mutated = individual.copy()
    for i in range(len(individual)):
        if rng.random() < mutation_rate:

```

```

        # 使用不同的变异策略
        mutation_type = rng.choice(['gaussian', 'uniform', 'bounda-
ry'])

        if mutation_type == 'gaussian':
            if i == 0:
                sigma = 30.0
            elif i == 1:
                sigma = (bounds[i, 1] - bounds[i, 0]) * 0.15
            else:
                sigma = (bounds[i, 1] - bounds[i, 0]) * 0.2

            mutated[i] += rng.normal(0, sigma)

        elif mutation_type == 'uniform':
            range_size = (bounds[i, 1] - bounds[i, 0]) * 0.3
            mutated[i] += rng.uniform(-range_size/2, range_size/2)

        else:
            if rng.random() < 0.5:
                mutated[i] = bounds[i, 0] + rng.random() * (bounds[i,
1] - bounds[i, 0]) * 0.1
            else:
                mutated[i] = bounds[i, 1] - rng.random() * (bounds[i,
1] - bounds[i, 0]) * 0.1

        # 边界约束
        if i == 0:
            mutated[i] = mutated[i] % 360.0
        else:
            mutated[i] = np.clip(mutated[i], bounds[i, 0], bounds[i,
1])

    return mutated

def solve_problem2_genetic(self, pop_size=150, n_generations=300, mu-
tation_rate=0.15,
                                crossover_rate=0.8, elite_ratio=0.1, ran-
dom_seed=1, plot_results=True,
                                use_parallel=True, n_processes=None):

    best_params, best_time = self.genetic_optimize(
        pop_size, n_generations, mutation_rate, crossover_rate,
        elite_ratio, random_seed, verbose=True, use_parallel=use_parallel,
        n_processes=n_processes
    )

    drone_flight_angle_opt, drone_speed_opt, release_time_opt, ex-
plode_delay_opt = best_params
    explode_time_opt = release_time_opt + explode_delay_opt

    # 输出最优解
    print("\n=== 遗传算法最优解 (问题 2) ===")
    print(f"无人机飞行角度  $\theta$  (度): {drone_flight_angle_opt:.2f}°")
    print(f"无人机飞行速度 v (m/s): {drone_speed_opt:.3f}")
    print(f"烟雾弹投放时刻 t_rel (s): {release_time_opt:.3f}")
    print(f"起爆延时  $\tau_{burst}$  (s): {explode_delay_opt:.3f}")
    print(f"起爆时刻 t_burst (s): {explode_time_opt:.3f}")
    print(f"最大有效遮蔽时长 (s): {best_time:.3f}")

```

```

        # 使用第一问的接口获取详细结果
        total_shielding_time, effective_periods, result_info =
self.solve_c_interference(
    f_angle=drone_flight_angle_opt,
    f_speed=drone_speed_opt,
    release_time=release_time_opt,
    explode_delay=explode_delay_opt,
    verbose=True
)

    # 绘制遮蔽时间线图
    if plot_results:
        self.plot_occlusion_timeline(result_info)

    return best_params, best_time, result_info

def plot_occlusion_timeline(self, result_info):

    explode_time = result_info['explode_time']

    dt = 0.05
    ts = np.arange(explode_time, explode_time + self.effective_time +
1e-9, dt)
    occl = []

    # 临时设置参数
    original_angle = self.f_angle
    self.f_angle = result_info['f_angle']

    try:
        target_points = self.get_true_points()

        c_center_func = self.calculate_c_center(
            explode_time, result_info['release_time'], re-
sult_info['f_speed']
        )

        for t in ts:
            m_pos = self.calculate_m_trajectory(t)
            c_center = c_center_func(t)

            if c_center is None or t > explode_time + self.effec-
tive_time:
                occl.append(0.0)
            else:
                is_effective = self.check_effective_shielding(m_pos,
c_center, target_points)
                occl.append(1.0 if is_effective else 0.0)

        finally:
            # 恢复原始角度
            self.f_angle = original_angle

    plt.figure(figsize=(10, 4))
    plt.plot(ts, occl, linewidth=2, color='red')
    plt.xlabel("Time t (s)")
    plt.ylabel("Occlusion Status (1=Fully Occluded)")
    plt.title("Complete Occlusion Timeline (Genetic Algorithm Optimal
Solution)")

```

```

plt.grid(True, alpha=0.3)
plt.ylim(-0.1, 1.1)
plt.show()

if __name__ == "__main__":
    model = GeneticCInterferenceModel()

    print(f"检测到 CPU 核心数: {cpu_count()}")
    print("开始并行遗传算法优化...")

    # 求解问题2 - 使用遗传算法 (启用并行计算)
    best_params, best_time, result_info = model.solve_problem2_genetic(
        pop_size=150,
        n_generations=300,
        mutation_rate=0.15,
        crossover_rate=0.8,
        elite_ratio=0.1,
        random_seed=42,
        plot_results=True,
        use_parallel=True,
        n_processes=None
    )

```

代码文件 4 问题三的求解(problem_3.py)

```

import numpy as np
import pandas as pd
import time
from problem_1 import CInterferenceModel

class OptimizedCInterferenceModelQ3(CInterferenceModel):
    def __init__(self):
        super().__init__()
        self.min_drone_speed = 70.0
        self.max_drone_speed = 140.0
        self.release_time_upper = 70.0
        self.explode_delay_upper = 15.0
        self.scan_dt = 0.01
        self.T_HARD_MAX = 120.0

    # 计算并集“完全遮蔽”总时长 (核心适应度)
    def calculate_single_bomb_occlusion_duration(self,
                                                drone_flight_angle_deg,
                                                drone_speed,
                                                release_time,
                                                explode_delay):

        # 基本约束
        if not (self.min_drone_speed <= drone_speed <=
self.max_drone_speed):
            return 0.0
        if release_time < 0.0 or explode_delay < 0.0:
            return 0.0

        # 起爆时刻
        explode_time = release_time + explode_delay

```

```

t0 = max(0.0, explode_time)
t1 = min(self.T_HARD_MAX, explode_time + self.effective_time)
if t1 <= t0:
    return 0.0

# 目标采样点 (第一问接口)
target_points = self.get_true_points()

original_angle = getattr(self, "drone_flight_angle", None)
self.drone_flight_angle = drone_flight_angle_deg
try:
    # 起爆后: 云心位于起爆点, 随时间以 3m/s 下沉
    cloud_func = self.calculate_c_center(explode_time=explode_time,
                                         release_time=release_time,
                                         drone_speed=drone_speed)

finally:
    self.drone_flight_angle = original_angle

# 扫描累计遮蔽时长
dt = self.scan_dt
total = 0.0
prev = False
seg_start = None

t = t0
while t <= t1 + 1e-12:
    # 导弹位置 (第一问接口)
    mpos = self.calculate_m_trajectory(t)

    # 云心位置
    cloud_center = None
    if explode_time <= t <= explode_time + self.effective_time:
        cloud_center = cloud_func(t)

    ok = False
    if cloud_center is not None:
        R = self.effective_radius
        ok_all_points = True
        for p in target_points:
            if not self.line_sphere_intersection(p, mpos, cloud_center, R):
                ok_all_points = False
                break
        ok = ok_all_points

    # 时间段累计
    if ok and not prev:
        seg_start = t
    elif (not ok) and prev and seg_start is not None:
        total += (t - seg_start)
        seg_start = None
    prev = ok
    t += dt

if prev and seg_start is not None:
    total += (t1 - seg_start)

return float(total)

```

```

def calculate_union_occlusion_duration_three(self,
                                             drone_flight_angle_deg,
                                             drone_speed,
                                             r1, r2, r3,
                                             d1, d2, d3):

    # 基本约束
    if not (self.min_drone_speed <= drone_speed <=
self.max_drone_speed):
        return 0.0
    if min(r1, r2, r3, d1, d2, d3) < 0.0:
        return 0.0

    r = np.array([r1, r2, r3], dtype=float)
    d = np.array([d1, d2, d3], dtype=float)
    idx = np.argsort(r)
    r = r[idx]
    d = d[idx]

    if r[1] < r[0] + 1.0 or r[2] < r[1] + 1.0:
        return 0.0

    e = r + d
    if np.any(e < r):
        return 0.0

    t0 = max(0.0, float(np.min(e)))
    t1 = min(self.T_HARD_MAX, float(np.max(e) + self.effective_time))
    if t1 <= t0:
        return 0.0

    target_points = self.get_true_points()

    original_angle = getattr(self, "drone_flight_angle", None)
    self.drone_flight_angle = drone_flight_angle_deg
    try:
        cloud_funcs = []
        for ri, ei in zip(r, e):

            f = self.calculate_c_center(explode_time=ei,
                                       release_time=ri,
                                       drone_speed=drone_speed)
            cloud_funcs.append((ri, ei, f))
    finally:
        self.drone_flight_angle = original_angle

    # 扫描累计并集时长
    dt = self.scan_dt
    total = 0.0
    prev = False
    seg_start = None

    t = t0
    while t <= t1 + 1e-12:
        # 导弹位置 (第一问接口)
        mpos = self.calculate_m_trajectory(t)

```

```

        centers = []
        for ri, ei, f in cloud_funcs:
            if ei <= t <= ei + self.effective_time:
                c = f(t)
                if c is not None:
                    centers.append(c)

        ok = False
        if centers:
            R = self.effective_radius
            ok_all_points = True
            for p in target_points:
                covered = False
                for c in centers:
                    if self.line_sphere_intersection(p, mpos, c, R):
                        covered = True
                        break
                if not covered:
                    ok_all_points = False
                    break
            ok = ok_all_points

        # 时间段累计
        if ok and not prev:
            seg_start = t
        elif (not ok) and prev and seg_start is not None:
            total += (t - seg_start)
            seg_start = None
        prev = ok
        t += dt

    if prev and seg_start is not None:
        total += (t1 - seg_start)

    return float(total)

# 粒子群优化
def pso_optimize_q3(self, n_particles=60, n_iters=120, random_seed=1,
verbose=True):
    start_time = time.time()

    if verbose:
        print("=== 问题 3: PSO 优化求解 (3 枚并集遮蔽时长) ===")
        print(f"粒子数量: {n_particles}")
        print(f"迭代次数: {n_iters}")
        print(f"随机种子: {random_seed}")
        print()

    rng = np.random.default_rng(random_seed)

    bounds = np.array([
        [0.0, 360.0], # 角度
        [self.min_drone_speed, self.max_drone_speed], # 速度
        [0.0, self.release_time_upper], # r1
        [0.0, self.release_time_upper], # r2
        [0.0, self.release_time_upper], # r3
        [0.0, self.explode_delay_upper], # d1
        [0.0, self.explode_delay_upper], # d2
        [0.0, self.explode_delay_upper], # d3
    ], dtype=float)

```

```

dim = bounds.shape[0]

# 初始化
pos = bounds[:, 0] + (bounds[:, 1] - bounds[:, 0]) * rng.random((n_particles, dim))
vel = np.zeros_like(pos)

# 适应度
def fitness(x):
    angle_deg, speed, r1, r2, r3, d1, d2, d3 = x
    # 修复: r 排序并同步重排 d, 拉开≥1s; 裁边界; 非负检查在 calculate 里也会兜
    r = np.array([r1, r2, r3], dtype=float)
    d = np.array([d1, d2, d3], dtype=float)
    idx = np.argsort(r)
    r = r[idx]
    d = d[idx]
    if r[1] < r[0] + 1.0:
        r[1] = r[0] + 1.0
    if r[2] < r[1] + 1.0:
        r[2] = r[1] + 1.0
    r = np.clip(r, 0.0, self.release_time_upper)
    d = np.clip(d, 0.0, self.explode_delay_upper)
    return self.calculate_union_occlusion_duration_three(angle_deg,
speed,
r[0], r[1], r[2],
d[0], d[1], d[2])

# 初始适应度
fit = np.array([fitness(p) for p in pos])
pbest = pos.copy()
pbest_fit = fit.copy()

# 全局最优
gi = np.argmax(fit)
gbest = pos[gi].copy()
gbest_fit = fit[gi]

# PSO 参数
w, c1, c2 = 0.72, 1.49, 1.49

if verbose:
    print(f"初始最优适应度: {gbest_fit:.3f}")

# PSO 主循环
for it in range(n_iters):
    r1 = rng.random((n_particles, dim))
    r2 = rng.random((n_particles, dim))

    # 更新速度、位置
    vel = w * vel + c1 * r1 * (pbest - pos) + c2 * r2 * (gbest -
pos)

    pos = pos + vel

    # 边界处理
    for dcol in range(dim):
        low, high = bounds[dcol]
        mask_low = pos[:, dcol] < low

```

```

        mask_high = pos[:, dcol] > high
        vel[mask_low, dcol] *= -0.5
        vel[mask_high, dcol] *= -0.5
        pos[:, dcol] = np.clip(pos[:, dcol], low, high)

    # 重新评估
    fit = np.array([fitness(p) for p in pos])

    # 个体最优
    improved = fit > pbest_fit
    pbest[improved] = pos[improved]
    pbest_fit[improved] = fit[improved]

    # 全局最优
    gi = np.argmax(fit)
    if fit[gi] > gbest_fit:
        gbest = pos[gi].copy()
        gbest_fit = fit[gi]

    if verbose and (it + 1) % 10 == 0:
        print(f"迭代 {it + 1}: 最优适应度 = {gbest_fit:.3f}")

    end_time = time.time()
    runtime = end_time - start_time

    if verbose:
        print(f"\n 优化完成, 用时: {runtime:.3f} 秒")

    return gbest, gbest_fit

def solve_problem3(self, n_particles=80, n_iters=150, random_seed=1,
                    plot_results=False, save_excel=True, excel_path="re-
sult1.xlsx"):
    best_params, best_time = self.pso_optimize_q3(n_particles, n_iters,
random_seed, verbose=True)
    angle_deg, speed, r1, r2, r3, d1, d2, d3 = best_params

    r = np.array([r1, r2, r3], dtype=float)
    d = np.array([d1, d2, d3], dtype=float)
    idx = np.argsort(r)
    r = r[idx]
    d = d[idx]
    if r[1] < r[0] + 1.0: r[1] = r[0] + 1.0
    if r[2] < r[1] + 1.0: r[2] = r[1] + 1.0
    r = np.clip(r, 0.0, self.release_time_upper)
    d = np.clip(d, 0.0, self.explode_delay_upper)
    e = r + d

    original_angle = getattr(self, "drone_flight_angle", None)
    self.drone_flight_angle = angle_deg
    try:
        plan_rows = []
        individual_durations = []
        for i in range(3):
            ri, ei, di = float(r[i]), float(e[i]), float(d[i])
            rel_pos = self.calculate_drone_position(ri, speed)
            cloud_func = self.calculate_c_center(ei, ri, speed)
            exp_pos = cloud_func(ei)

```

```

        single_duration = self.calculate_single_bomb_occlusion_duration(
            angle_deg, speed, ri, di
        )
        individual_durations.append(single_duration)

    plan_rows.append({
        "drone_heading_deg": angle_deg,
        "drone_speed_mps": speed,
        "bomb_id": i + 1,
        "release_time_s": ri,
        "release_x_m": rel_pos[0],
        "release_y_m": rel_pos[1],
        "release_z_m": rel_pos[2],
        "explode_time_s": ei,
        "explode_x_m": exp_pos[0],
        "explode_y_m": exp_pos[1],
        "explode_z_m": exp_pos[2],
        "individual_duration_s": single_duration,
    })
finally:
    self.drone_flight_angle = original_angle

# 打印最优解
print("\n=== PSO 最优解 (问题 3) ===")
print(f"无人机飞行角度  $\theta$  (度): {angle_deg:.2f}°")
print(f"无人机飞行速度  $v$  (m/s): {speed:.3f}")
for i in range(3):
    print(f"弹{i+1}: 投放  $r[i+1]=\{r[i]:.3f\}$ s, 起爆  $e[i+1]=\{e[i]:.3f\}$ s  
(延时  $d[i+1]=\{d[i]:.3f\}$ s), 单独遮蔽时长: {individual_durations[i]:.3f}s")
    print(f"三朵云并集\完全遮蔽\总时长: {best_time:.3f} s")

if save_excel:
    # 构造符合格式要求的数据
    output_rows = []
    for i in range(3):
        row_data = [
            angle_deg,
            speed,
            i + 1,
            plan_rows[i]["release_x_m"],
            plan_rows[i]["release_y_m"],
            plan_rows[i]["release_z_m"],
            plan_rows[i]["explode_x_m"],
            plan_rows[i]["explode_y_m"],
            plan_rows[i]["explode_z_m"],
            individual_durations[i]
        ]
        output_rows.append(row_data)

    output_rows.append([np.nan] * 10)

    note_row = ["注: 以 x 轴为正向, 逆时针方向为正, 取值 0~360 (度)。"] + [np.nan] * 9
    output_rows.append(note_row)

    columns = [
        "无人机运动方向",

```

```

        "无人机运动速度 (m/s)",
        "烟幕干扰弹编号",
        "烟幕干扰弹投放点的 x 坐标 (m)",
        "烟幕干扰弹投放点的 y 坐标 (m)",
        "烟幕干扰弹投放点的 z 坐标 (m)",
        "烟幕干扰弹起爆点的 x 坐标 (m)",
        "烟幕干扰弹起爆点的 y 坐标 (m)",
        "烟幕干扰弹起爆点的 z 坐标 (m)",
        "有效干扰时长 (s)"
    ]

    df_result = pd.DataFrame(output_rows, columns=columns)

    # 保存到 Excel
    df_result.to_excel(excel_path, index=False,
sheet_name="Sheet1")
    print(f"已保存到: {excel_path}")

    detail = {
        "angle_deg": angle_deg,
        "speed": speed,
        "r": r.copy(),
        "d": d.copy(),
        "e": e.copy()
    }

    return np.array([angle_deg, speed, r[0], r[1], r[2], d[0], d[1],
d[2]], dtype=float), best_time, detail

if __name__ == "__main__":
    # 创建优化模型实例 (问题3)
    model = OptimizedCInterferenceModelQ3()
    # 求解问题3 (PSO 优化 + 导出 Excel)
    best_params, best_time, detail = model.solve_problem3(
        n_particles=50,
        n_iters=80,
        random_seed=1,
        plot_results=False,
        save_excel=True,
        excel_path="result1.xlsx"
    )

```

代码文件 5 PSO+模拟退火优化求解问题四(problem_4.py)

```

import numpy as np
import pandas as pd
import math, time, random
import matplotlib.pyplot as plt

g = 9.8
MISSILE_V = 300.0
CLOUD_R = 10.0
CLOUD_T = 20.0
CLOUD_SINK = 3.0
VERIFY_DT = 0.01

```

```

CYL_CENTER = np.array([0.0, 200.0, 0.0])
CYL_R = 7.0
CYL_H = 10.0
FAKE = np.array([0.0, 0.0, 0.0])

MISSILES = {
    "M1": np.array([20000.0, 0.0, 2000.0]),
}
TARGET_MISSILE = "M1"

UAVS = {
    "FY1": np.array([17800.0, 0.0, 1800.0]),
    "FY2": np.array([12000.0, 1400.0, 1400.0]),
    "FY3": np.array([6000.0, -3000.0, 700.0]),
}

V_MIN, V_MAX = 70.0, 140.0

ALPHA_MIN, ALPHA_MAX = -180.0, 180.0

TE_MIN, TE_MAX = 0.5, 35.0

# 输出
EXCEL_PATH = "q4_pso_sa.xlsx"
PNG_VERIFY = "q4_verify_timeline.png"
PNG_PSO = "q4_pso_sa_timeline.png"
def unit(v):
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

def missile_pos(mname, t):
    m0 = MISSILES[mname]
    return m0 + unit(FAKE - m0) * MISSILE_V * t

def cylinder_surface_points(n_angle=36, heights=(0.0, CYL_H)):
    ang = np.linspace(0, 2*np.pi, n_angle, endpoint=False)
    pts = []
    for hz in heights:
        for a in ang:
            x = CYL_CENTER[0] + CYL_R * np.cos(a)
            y = CYL_CENTER[1] + CYL_R * np.sin(a)
            z = CYL_CENTER[2] + hz
            pts.append([x, y, z])
    return np.array(pts)

SURF_PTS = cylinder_surface_points()

def seg_sphere_intersect(M, P, C, R):
    d = P - M
    f = M - C
    dd = float(np.dot(d, d))
    if dd == 0:
        return np.linalg.norm(M - C) <= R
    u = -float(np.dot(d, f)) / dd
    if u < 0: closest = M
    elif u > 1: closest = P
    else: closest = M + u * d
    return np.linalg.norm(closest - C) <= R

```

```

def full_occlusion_at_time(mname, t, C):
    M = missile_pos(mname, t)
    for P in SURF_PTS:
        if not seg_sphere_intersect(M, P, C, CLOUD_R):
            return False
    return True

def merge_intervals(intervals):
    if not intervals: return []
    xs = sorted(intervals)
    out = [list(xs[0])]
    for s,e in xs[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s,e])
    return [(float(s), float(e)) for s,e in out]

def union_length(intervals):
    m = merge_intervals(intervals)
    return sum(e - s for s, e in m)

def verify_20s(mname, te, C, dt=VERIFY_DT):
    times = np.arange(te, te + CLOUD_T + 1e-9, dt)
    ok = []; intervals = []; start = None
    for t in times:
        Ct = np.array([C[0], C[1], C[2] - CLOUD_SINK * (t - te)])
        ok.append(full_occlusion_at_time(mname, t, Ct))
    for i, t in enumerate(times):
        end_of_scan = (i == len(times)-1)
        if ok[i] and start is None:
            start = t
        if (not ok[i] and start is not None) or (end_of_scan and ok[i]):
            end = t
            if end - start >= dt * 0.5:
                intervals.append((float(start), float(end)))
            start = None
    return merge_intervals(intervals)

def decision_to_C(uav, v, alpha_deg, release, te):
    U0 = UAVS[uav]; U0_xy = U0[:2]; Zu = U0[2]
    a = math.radians(alpha_deg)
    Cxy = U0_xy + v * te * np.array([math.cos(a), math.sin(a)])
    if te < release:
        return None
    Cz = Zu - 0.5 * g * (te - release) * (te - release)
    return np.array([Cxy[0], Cxy[1], Cz], dtype=float)

def quick_angle_gate(mname, te, C):
    A_AXIS = np.array([0.0, 200.0, 5.0])
    Mte = missile_pos(mname, te)
    d = np.linalg.norm(C - Mte)
    if d <= 1e-9: return False
    theta = math.sqrt(CYL_R**2+(CYL_H/2.0)**2) / np.linalg.norm(A_AXIS -
Mte)
    phi = CLOUD_R / d
    return phi >= 0.8 * theta

```

```

def fitness_of_particle(x):
    uavs = ["FY1", "FY2", "FY3"]
    tracks = {}
    feas = True
    for i, u in enumerate(uavs):
        v = float(np.clip(x[4*i+0], V_MIN, V_MAX))
        a = float(np.clip(x[4*i+1], ALPHA_MIN, ALPHA_MAX))
        te = float(np.clip(x[4*i+3], TE_MIN, TE_MAX))
        r = float(np.clip(x[4*i+2], 0.0, te-1e-6)) # release ≤ te-ε
        C = decision_to_C(u, v, a, r, te)
        if C is None:
            tracks[u] = []; feas = False; continue
        if not quick_angle_gate(TARGET_MISSILE, te, C):
            tracks[u] = []; feas = False; continue
        intervals = verify_20s(TARGET_MISSILE, te, C, dt=VERIFY_DT)
        tracks[u] = intervals
    all_ivs=[]; [all_ivs.extend(tracks[u]) for u in ["FY1", "FY2", "FY3"]]
    L = union_length(merge_intervals(all_ivs))
    if not feas:
        L *= 0.5 # 轻微惩罚不可行
    return L, tracks

def draw_timeline(tracks, union_track, out_png, title):
    fig, ax = plt.subplots(figsize=(12, 4))
    levels = {"FY1": 3, "FY2": 2, "FY3": 1, "UNION": 0}
    colors = {"FY1": "tab:blue", "FY2": "tab:orange", "FY3": "tab:green", "UNION": "tab:red"}
    for u in ["FY1", "FY2", "FY3"]:
        for s, e in tracks.get(u, []):
            ax.hlines(levels[u], s, e, colors=colors[u], linewidth=6, label=u)
    for s, e in union_track:
        ax.hlines(levels["UNION"], s, e, colors=colors["UNION"], linewidth=8, label="UNION")
    handles, labels = ax.get_legend_handles_labels()
    uniq = dict(zip(labels, handles))
    ax.legend(uniq.values(), uniq.keys(), ncol=4, fontsize=8, title="Tracks")
    if union_track:
        xmin = min(s for s, _ in union_track); xmax = max(e for _, e in union_track)
        ax.set_xlim(xmin-1, xmax+1)
        ax.set_yticks([0, 1, 2, 3]); ax.set_yticklabels(["UNION", "FY3", "FY2", "FY1"])
        ax.set_xlabel("Time (s)"); ax.set_title(title)
        ax.grid(True, linestyle="--", linewidth=0.5, alpha=0.6)
        plt.tight_layout(); plt.savefig(out_png, dpi=180); plt.show()
        print("Saved timeline:", out_png)

# Step 1: 验证给定解
def verify_given():
    given = {
        "FY1": dict(v=104.9135664, a=5.953461572, r=0.603319112, te=1.5,
                    C=np.array([17956.52157, 16.32255175, 1796.060221])),
        "FY2": dict(v=140.0, a=-73.97470199, r=4.285870532, te=10.0,
                    C=np.array([12386.48646, 54.40414126, 1240.00875])),
        "FY3": dict(v=138.1915533, a=89.13740422, r=18.15293604, te=22.5,
                    C=np.array([6046.809317, 108.9575821, 607.4048713])),
    }
    rows = []; tracks = {}

```

```

print("=== Step 1: Verify Given Solution ===")
for u,par in given.items():
    v,a,r,te = par["v"], par["a"], par["r"], par["te"]
    C_est = decision_to_C(u, v, a, r, te)
    ctrl_ok = C_est is not None
    # 与给定 c 的一致性 (水平必然一致; 垂直按自由落体公式一致)
    err = np.linalg.norm(C_est - par["C"]) if ctrl_ok else float("inf")
    ivs = [] if not ctrl_ok else verify_20s(TARGET_MISSILE, te, C_est,
dt=VERIFY_DT) \
        if quick_angle_gate(TARGET_MISSILE, te, C_est) else
[]
    tracks[u] = ivs
    cov = union_length(ivs)
    print(f" {u}: te={te:.2f}s v={v:.2f} alpha={a:.2f}° re-
lease={r:.2f}s"
        f" |C_err|={0 if err==float('inf') else err:.3f}
cover={cov:.2f}s ivs={ivs}")
    rows.append(dict(UAV=u, v=v, alpha_deg=a, release_time_s=r, ex-
plode_time_s=te,
        explode_x=C_est[0], explode_y=C_est[1], ex-
plode_z=C_est[2],
        cover_len=cov, intervals=str(ivs)))
    all_ivs=[]; [all_ivs.extend(tracks[u]) for u in ["FY1","FY2","FY3"]]
    union_ivs=merge_intervals(all_ivs); union_len=union_length(union_ivs)
    print(f"UNION covered length = {union_len:.2f} s")
    df = pd.DataFrame(rows)
    try:
        with pd.ExcelWriter(EXCEL_PATH, engine="openpyxl") as w:
            df.to_excel(w, index=False, sheet_name="verify_given")
            print("Saved Excel:", EXCEL_PATH)
    except Exception as e:
        csv = EXCEL_PATH.replace(".xlsx", ".csv")
        df.to_csv(csv, index=False); print("Saved CSV:", csv, "|", e)
    draw_timeline(tracks, union_ivs, PNG_VERIFY, "Q4 Verify (Given)")

    # 保存给定解的 result2 格式
    given_x = [given["FY1"]["v"], given["FY1"]["a"], given["FY1"]["r"],
given["FY1"]["te"],
        given["FY2"]["v"], given["FY2"]["a"], given["FY2"]["r"],
given["FY2"]["te"],
        given["FY3"]["v"], given["FY3"]["a"], given["FY3"]["r"],
given["FY3"]["te"]]
    save_final_result_format(given_x, tracks, "result2_given_solu-
tion.xlsx")

    # 返回作为 PSO 种子
    seed = given_x
    return seed, union_len

# 辅助函数: 计算投放点和格式化输出
def calculate_drop_position(uav, v, alpha_deg, release):
    """计算烟幕干扰弹投放点坐标"""
    U0 = UAVS[uav]
    a = math.radians(alpha_deg)
    # 投放点 = 初始位置 + v * release * [cos(α), sin(α), 0]
    drop_x = U0[0] + v * release * math.cos(a)
    drop_y = U0[1] + v * release * math.sin(a)
    drop_z = U0[2] # 投放时高度不变
    return np.array([drop_x, drop_y, drop_z])

```

```

def format_direction_angle(alpha_deg):
    """将角度转换为0-360度格式"""
    angle = alpha_deg % 360
    return angle if angle >= 0 else angle + 360

def save_final_result_format(best_x, best_tracks, filename="result2.xlsx"):
    """按照result2格式保存最终结果"""
    uavs = ["FY1", "FY2", "FY3"]
    rows = []

    for i, uav in enumerate(uavs):
        # 提取参数
        v = best_x[4*i + 0]
        alpha = best_x[4*i + 1]
        release = best_x[4*i + 2]
        te = best_x[4*i + 3]

        # 计算投放点和爆炸点
        drop_pos = calculate_drop_position(uav, v, alpha, release)
        explode_pos = decision_to_C(uav, v, alpha, release, te)

        # 计算该无人机的有效干扰时长
        intervals = best_tracks.get(uav, [])
        interference_duration = union_length(intervals)

        # 格式化角度为0-360度
        direction_angle = format_direction_angle(alpha)

        row = {
            '无人机编号': uav,
            '无人机运动方向': f"{direction_angle:.2f}",
            '无人机运动速度 (m/s)': f"{v:.2f}",
            '烟幕干扰弹投放点的x坐标 (m)': f"{drop_pos[0]:.2f}",
            '烟幕干扰弹投放点的y坐标 (m)': f"{drop_pos[1]:.2f}",
            '烟幕干扰弹投放点的z坐标 (m)': f"{drop_pos[2]:.2f}",
            '烟幕干扰弹起爆点的x坐标 (m)': f"{explode_pos[0]:.2f}",
            '烟幕干扰弹起爆点的y坐标 (m)': f"{explode_pos[1]:.2f}",
            '烟幕干扰弹起爆点的z坐标 (m)': f"{explode_pos[2]:.2f}",
            '有效干扰时长 (s)': f"{interference_duration:.2f}"
        }
        rows.append(row)

    # 添加空行和注释行
    rows.append({col: "" for col in rows[0].keys()})
    comment_row = {col: "" for col in rows[0].keys()}
    comment_row['无人机运动方向'] = "注：以x轴为正向，逆时针方向为正，取值0~360(度)。"
    rows.append(comment_row)

    df = pd.DataFrame(rows)

    try:
        df.to_excel(filename, index=False, engine='openpyxl')
        print(f"已保存最终结果到: {filename}")
        print("\n=== 最终解参数 ===")
    
```

```

for i, uav in enumerate(uavs):
    v = best_x[4*i + 0]
    alpha = best_x[4*i + 1]
    release = best_x[4*i + 2]
    te = best_x[4*i + 3]
    direction_angle = format_direction_angle(alpha)
    intervals = best_tracks.get(uav, [])
    interference_duration = union_length(intervals)

    print(f"{uav}: 速度={v:.2f}m/s, 方向={direction_angle:.2f}°, "
          f"投放时间={release:.2f}s, 爆炸时间={te:.2f}s, "
          f"干扰时长={interference_duration:.2f}s")

except Exception as e:
    csv_filename = filename.replace('.xlsx', '.csv')
    df.to_csv(csv_filename, index=False, encoding='utf-8-sig')
    print(f"Excel 保存失败, 已保存为 CSV: {csv_filename} | 错误: {e}")

return df

# Step 2: PSO + 模拟退火
def clamp_angle(a):
    # wrap to [-180,180]
    while a > 180: a -= 360
    while a < -180: a += 360

```

代码文件 6 启发式优化求解问题四(problem_4_equation.py)

```

import numpy as np
import pandas as pd
import math
import time
import itertools
from problem_1 import CInterferenceModel

class OptimizedCInterferenceModelQ4(CInterferenceModel):

    def __init__(self):
        super().__init__()

        self.min_drone_speed = 70.0
        self.max_drone_speed = 140.0

        # 多导弹配置
        self.missiles = {
            "M1": np.array([20000.0, 0.0, 2000.0]),
            "M2": np.array([19000.0, 600.0, 2100.0]),
            "M3": np.array([18000.0, -600.0, 1900.0]),
        }

        # 多无人机配置
        self.uavs = {
            "FY1": np.array([17800.0, 0.0, 1800.0]),
            "FY2": np.array([12000.0, 1400.0, 1400.0]),
            "FY3": np.array([6000.0, -3000.0, 700.0]),
        }

        self.target_axis = np.array([0.0, 200.0, 5.0])

```

```

self.target_missile = "M1"

self.verify_dt = 0.25

self.explode_time_start = 0.5
self.explode_time_step = 0.5

self.top_k_per_uav = 40

self.excel_path = "heuristic_candidates_q4.xlsx"

def normalize_vector(self, v):
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

def calculate_missile_position(self, missile_name, t):
    missile_init = self.missiles[missile_name]
    direction = self.normalize_vector(self.fake_target - missile_init)
    return missile_init + direction * self.m_speed * t

def get_cylinder_surface_points(self, n_angle=24, heights=None):
    if heights is None:
        heights = (0.0, self.target_height/2.0, self.target_height)

    angles = np.linspace(0, 2*np.pi, n_angle, endpoint=False)
    points = []

    for height in heights:
        for angle in angles:
            x = self.true_center[0] + self.target_radius * np.cos(angle)
            y = self.true_center[1] + self.target_radius * np.sin(angle)
            z = self.true_center[2] + height
            points.append([x, y, z])

    return np.array(points)

def check_segment_sphere_intersection(self, point_start, point_end,
sphere_center, sphere_radius):
    d = point_end - point_start
    f = point_start - sphere_center
    dd = float(np.dot(d, d))

    if dd == 0:
        return np.linalg.norm(point_start - sphere_center) <=
sphere_radius

    u = -float(np.dot(d, f)) / dd
    if u < 0:
        closest = point_start
    elif u > 1:
        closest = point_end
    else:
        closest = point_start + u * d

    return np.linalg.norm(closest - sphere_center) <= sphere_radius

def check_full_occlusion_at_time(self, missile_name, t, cloud_center):
    missile_pos = self.calculate_missile_position(missile_name, t)
    surface_points = self.get_cylinder_surface_points()

```

```

        for point in surface_points:
            if not self.check_segment_sphere_intersection(
                missile_pos, point, cloud_center, self.effective_radius
            ):
                return False
        return True

    def check_quick_angle_gate(self, missile_name, explode_time,
cloud_center):
        missile_pos = self.calculate_missile_position(missile_name, ex-
plode_time)
        distance = np.linalg.norm(cloud_center - missile_pos)

        if distance <= 1e-9:
            return False

        # 圆柱半对角 / 视距 ≈ 目标张角; 云球张角 ≈ R/d
        target_angle = math.sqrt(
            self.target_radius**2 + (self.target_height/2.0)**2
        ) / np.linalg.norm(self.target_axis - missile_pos)
        cloud_angle = self.effective_radius / distance

        return cloud_angle >= 0.8 * target_angle

    def merge_intervals(self, intervals):
        if not intervals:
            return []

        sorted_intervals = sorted(intervals)
        merged = [list(sorted_intervals[0])]

        for start, end in sorted_intervals[1:]:
            if start <= merged[-1][1]:
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])

        return [(float(s), float(e)) for s, e in merged]

    def calculate_union_length(self, intervals):
        merged = self.merge_intervals(intervals)
        return sum(end - start for start, end in merged)

    def verify_20s_occlusion(self, missile_name, explode_time, cloud_cen-
ter, dt=None):
        if dt is None:
            dt = self.verify_dt

        times = np.arange(explode_time, explode_time + self.effective_time
+ 1e-9, dt)
        occlusion_status = []
        intervals = []
        start_time = None

        for t in times:
            cloud_pos = np.array([
                cloud_center[0],
                cloud_center[1],
                cloud_center[2] - self.smoke_sink_speed * (t - explode_time)
            ])

```

```

    ])
    occlusion_status.append(self.check_full_occlusion_at_time(mis-
sile_name, t, cloud_pos))

    for i, t in enumerate(times):
        if occlusion_status[i] and start_time is None:
            start_time = t

        end_of_scan = (i == len(times) - 1)
        if (end_of_scan and occlusion_status[i]) or (not occlusion_sta-
tus[i] and start_time is not None):
            end_time = t
            if end_time - start_time >= dt * 0.5:
                intervals.append((float(start_time), float(end_time)))
            start_time = None

    return self.merge_intervals(intervals)

def solve_quadratic_equation(self, a, b, c):

    if abs(a) < 1e-12:
        if abs(b) < 1e-12:
            return []
        return [-c / b]

    discriminant = b*b - 4*a*c
    if discriminant < -1e-12:
        return []
    if discriminant < 0:
        discriminant = 0.0

    sqrt_disc = math.sqrt(discriminant)
    return [(-b - sqrt_disc) / (2*a), (-b + sqrt_disc) / (2*a)]

def calculate_lambda_from_ring(self, uav_pos_xy, missile_pos_xy, di-
rection, radius):
    w = missile_pos_xy - uav_pos_xy
    a = float(np.dot(direction, direction))
    b = 2.0 * float(np.dot(direction, w))
    c = float(np.dot(w, w)) - radius*radius

    roots = self.solve_quadratic_equation(a, b, c)
    return [lam for lam in roots if 0.0 <= lam <= 1.0]

def generate_lambda_candidates_analytic(self, uav_name, missile_name,
explode_time):
    uav_pos_xy = self.uavs[uav_name][:2]
    missile_pos_full = self.calculate_missile_position(missile_name,
explode_time)
    missile_pos_xy = missile_pos_full[:2]
    direction = (self.target_axis - missile_pos_full)[:2]

    if np.linalg.norm(direction) < 1e-9:
        return []

    radius_min = self.min_drone_speed * explode_time
    radius_max = self.max_drone_speed * explode_time

    roots = []
    for radius in [radius_min, radius_max]:

```

```

        roots.extend(self.calculate_lambda_from_ring(uav_pos_xy, mis-
sile_pos_xy, direction, radius))

    roots.extend([0.0, 1.0])
    roots = sorted(set([max(0.0, min(1.0, float(x))) for x in roots]))

    candidates = []
    for i in range(len(roots)-1):
        lambda1, lambda2 = roots[i], roots[i+1]
        mid = 0.5 * (lambda1 + lambda2)
        cloud_xy = missile_pos_xy + mid * direction
        distance = np.linalg.norm(cloud_xy - uav_pos_xy)

        if radius_min - 1e-6 <= distance <= radius_max + 1e-6:
            mid_clipped = float(np.clip(mid, 0.02, 0.98))
            candidates.append(mid_clipped)

    for lam in (self.calculate_lambda_from_ring(uav_pos_xy, mis-
sile_pos_xy, direction, radius_min) +
               self.calculate_lambda_from_ring(uav_pos_xy, mis-
sile_pos_xy, direction, radius_max)):
        lam_clipped = float(np.clip(lam, 0.02, 0.98))
        if 0.0 <= lam_clipped <= 1.0:
            candidates.append(lam_clipped)

    candidates = sorted(set([float(x) for x in candidates]))
    return candidates

    def calculate_candidate_cloud_center(self, missile_name, explode_time,
lambda_param):
        missile_pos = self.calculate_missile_position(missile_name, ex-
plode_time)
        return missile_pos + lambda_param * (self.target_axis - mis-
sile_pos)

    def plan_controls_for_cloud_center(self, uav_name, explode_time,
cloud_center):

        uav_pos = self.uavs[uav_name]
        uav_pos_xy = uav_pos[:2]
        cloud_center_xy = cloud_center[:2]

        distance_xy = np.linalg.norm(cloud_center_xy - uav_pos_xy)
        drone_speed = distance_xy / explode_time if explode_time > 1e-9
else float("inf")

        if not (self.min_drone_speed <= drone_speed <=
self.max_drone_speed):
            return None

        dx, dy = (cloud_center_xy - uav_pos_xy)
        flight_angle = math.atan2(dy, dx)

        uav_height = uav_pos[2]
        if cloud_center[2] > uav_height:
            return None

        fall_time_squared = 2.0 * (uav_height - cloud_center[2]) / self.g
        if fall_time_squared < 0:
            return None

```

```

        fall_time = math.sqrt(fall_time_squared)
        if explode_time < fall_time:
            return None

        release_time = explode_time - fall_time
        delay = 0.0 if release_time >= 0.0 else -release_time
        release_time = max(0.0, release_time)

        return {
            'drone_speed': drone_speed,
            'flight_angle': flight_angle,
            'release_time': release_time,
            'delay': delay
        }

    def generate_candidates_for_uav(self, uav_name, missile_name):
        max_time = np.linalg.norm(self.missiles[missile_name] -
self.fake_target) / self.m_speed
        candidates = []

        explode_time_grid = np.arange(self.explode_time_start, max_time,
self.explode_time_step)

        for explode_time in explode_time_grid:
            lambda_list = self.generate_lambda_candidates_analytic(uav_name, missile_name, explode_time)

            for lambda_param in lambda_list:
                cloud_center = self.calculate_candidate_cloud_center(missile_name, explode_time, lambda_param)
                controls = self.plan_controls_for_cloud_center(uav_name, explode_time, cloud_center)

                if controls is None:
                    continue

                if not self.check_quick_angle_gate(missile_name, explode_time, cloud_center):
                    continue

                intervals = self.verify_20s_occlusion(missile_name, explode_time, cloud_center, dt=self.verify_dt)
                if not intervals:
                    continue

                score = self.calculate_union_length(intervals)

                candidates.append({
                    'uav': uav_name,
                    'missile': missile_name,
                    'explode_time': float(explode_time),
                    'lambda': float(lambda_param),
                    'cloud_center_x': float(cloud_center[0]),
                    'cloud_center_y': float(cloud_center[1]),
                    'cloud_center_z': float(cloud_center[2]),
                    'drone_speed': float(controls["drone_speed"]),
                    'flight_angle': float(controls["flight_angle"]),
                    'release_time': float(controls["release_time"]),
                    'delay': float(controls["delay"]),
                })

```

```

        'intervals': intervals,
        'score': float(score)
    })

    candidates.sort(key=lambda c: c["score"], reverse=True)
    return candidates[:self.top_k_per_uav]

def search_best_combination(self, candidates_by_uav):
    uav_names = ["FY1", "FY2", "FY3"]
    candidate_lists = [candidates_by_uav[uav_name] for uav_name in
uav_names]

    best_combination = None
    best_total_time = -1.0

    for combination in itertools.product(*candidate_lists):
        all_intervals = []
        for candidate in combination:
            all_intervals.extend(candidate["intervals"])

        total_time = self.calculate_union_length(all_intervals)
        if total_time > best_total_time:
            best_total_time = total_time
            best_combination = combination

    return best_combination, best_total_time

def solve_problem4(self, save_excel=True, verbose=True):
    start_time = time.time()

    if verbose:
        print("=== 问题 4: 多导弹多无人机烟幕干扰优化 ===")
        print("功能: 生成优秀搜索起始点")
        print(f"目标导弹: {self.target_missile}")
        print(f"参与无人机: FY1, FY2, FY3")
        print(f"验证时间步长: {self.verify_dt}s, 每架无人机保留候选:
{self.top_k_per_uav}")
        print()

    # 1) 为三架无人机生成候选
    candidates_by_uav = {}
    total_candidates = 0

    for uav_name in ["FY1", "FY2", "FY3"]:
        candidates = self.generate_candidates_for_uav(uav_name,
self.target_missile)
        candidates_by_uav[uav_name] = candidates
        total_candidates += len(candidates)

        if verbose:
            print(f"[{uav_name}] 候选数: {len(candidates)}")

    if any(len(candidates_by_uav[uav]) == 0 for uav in candi-
dates_by_uav):
        print("警告: 某些无人机无可行候选, 请调整参数")
        return None, None, None

```

```

        if verbose:
            print(f"总候选数: {total_candidates}, 开始组合搜索...")

        # 2) 搜索最佳组合
        best_combination, best_time = self.search_best_combination(candidates_by_uav)
        end_time = time.time()
        runtime = end_time - start_time

        if verbose:
            print("\n=== 启发点生成结果 ===")
            print(f"当前最优遮蔽时长 (步长为{self.verify_dt}, 粗筛): {best_time:.3f}s")
            print("推荐启发点:")

            for candidate in best_combination:
                print(f" {candidate['uav']}: 起爆时间={candidate['explode_time']:.2f}s, "
                    f"速度={candidate['drone_speed']:.2f}m/s, "
                    f"航向角={math.degrees(candidate['flight_angle']):.2f}°, "
                    f"投放时间={candidate['release_time']:.2f}s, "
                    f"延时={candidate['delay']:.2f}s, "
                    f"起爆位置=({candidate['cloud_center_x']:.1f}, "
                    f"{candidate['cloud_center_y']:.1f}, "
                    f"{candidate['cloud_center_z']:.1f}), "
                    f"单独遮蔽时长={self.calculate_union_length(candidate['intervals']):.2f}s")

            print(f"程序运行时长: {runtime:.3f} s")
            print(f"扫描候选总数: {total_candidates}")

        # 3) 导出启发点到Excel
        if save_excel:
            rows = []
            for candidate in best_combination:
                rows.append({
                    "UAV": candidate["uav"],
                    "Missile": candidate["missile"],
                    "heading_deg": math.degrees(candidate["flight_angle"]),
                    "speed_mps": candidate["drone_speed"],
                    "release_time_s": candidate["release_time"],
                    "explode_time_s": candidate["explode_time"],
                    "delay_s": candidate["delay"],
                    "explode_x": candidate["cloud_center_x"],
                    "explode_y": candidate["cloud_center_y"],
                    "explode_z": candidate["cloud_center_z"],
                    "intervals": str(candidate["intervals"])
                })

            df = pd.DataFrame(rows)
            try:
                with pd.ExcelWriter(self.excel_path, engine="openpyxl") as writer:
                    df.to_excel(writer, index=False, sheet_name="Heuristic_Points")
            if verbose:

```

```

        print(f"启发点已保存: {self.excel_path}")
    except Exception as e:
        csv_path = self.excel_path.replace(".xlsx", ".csv")
        df.to_csv(csv_path, index=False)
        if verbose:
            print(f"已保存CSV: {csv_path}")

    # 输出并集信息
    if verbose:
        all_intervals = []
        for candidate in best_combination:
            all_intervals.extend(candidate["intervals"])
        merged_intervals = self.merge_intervals(all_intervals)
        print(f"并集时段: {merged_intervals}")

    result_info = {
        'runtime': runtime,
        'total_candidates': total_candidates,
        'candidates_by_uav': candidates_by_uav
    }

    return best_combination, best_time, result_info

if __name__ == "__main__":
    model = OptimizedCInterferenceModelQ4()

    best_combination, best_time, result_info = model.solve_problem4(
        save_excel=True,
        verbose=True
    )

```

代码文件 7 第五问解空间优化阶段(problem_5.py)

```

import numpy as np
import pandas as pd
import math, time, itertools, collections
from dataclasses import dataclass

g = 9.8
MISSILE_V = 300.0
CLOUD_R = 10.0
CLOUD_T = 20.0
CLOUD_SINK = 3.0

TARGET_CENTER = np.array([0.0, 200.0, 0.0])
TARGET_R = 7.0
TARGET_H = 10.0
FAKE = np.array([0.0, 0.0, 0.0])

MISSILES = {
    "M1": np.array([20000.0, 0.0, 2000.0]),
    "M2": np.array([19000.0, 600.0, 2100.0]),
    "M3": np.array([18000.0, -600.0, 1900.0]),
}
UAVS = {
    "FY1": np.array([17800.0, 0.0, 1800.0]),
    "FY2": np.array([12000.0, 1400.0, 1400.0]),
    "FY3": np.array([6000.0, -3000.0, 700.0]),
}

```

```

    "FY4": np.array([11000.0, 2000.0, 1800.0]),
    "FY5": np.array([13000.0, -2000.0, 1300.0]),
}

VERIFY_DT = 0.25
EXPLODE_T0 = 0.5
EXPLODE_D = 0.5
TOPK_PER_PAIR = 25
SPEED_MIN, SPEED_MAX = 70.0, 140.0

HEADING_BIN_DEG = 1.0
SPEED_BIN_MPS = 1.0

MAX_SHOTS_PER_UAV = 3
MIN_RELEASE_GAP = 1.0

CLUSTER_ENUM_TOP_L = 10
# Excel 输出
RESULT_XLSX = "result3.xlsx"

def norm(v):
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

def missile_pos(mname, t):
    m0 = MISSILES[mname]
    dirv = norm(FAKE - m0)
    return m0 + dirv * MISSILE_V * t

def cylinder_surface(n_angle=24, heights=None):
    if heights is None:
        heights = (0.0, TARGET_H/2.0, TARGET_H)
    ang = np.linspace(0, 2*np.pi, n_angle, endpoint=False)
    P = []
    for h in heights:
        for a in ang:
            x = TARGET_CENTER[0] + TARGET_R*np.cos(a)
            y = TARGET_CENTER[1] + TARGET_R*np.sin(a)
            z = TARGET_CENTER[2] + h
            P.append([x,y,z])
    return np.asarray(P, dtype=float)

CYL_SAMPLES = cylinder_surface(n_angle=24, heights=(0.0, TARGET_H/2.0,
TARGET_H))

def seg_sphere_intersect(A, B, C, R):
    d = B - A
    f = A - C
    dd = float(np.dot(d, d))
    if dd == 0:
        return np.linalg.norm(A - C) <= R
    u = -float(np.dot(d, f)) / dd
    if u < 0: P = A
    elif u > 1: P = B
    else: P = A + u*d
    return np.linalg.norm(P - C) <= R

def angle_gate(mname, te, cloud_xyz, relax=0.8):
    M = missile_pos(mname, te)

```

```

    dist = np.linalg.norm(cloud_xyz - M)
    if dist <= 1e-12: return False
    target_ang = math.sqrt(TARGET_R**2 + (TARGET_H/2.0)**2) /
np.linalg.norm(TARGET_CENTER + [0,0,TARGET_H/2.0] - M)
    cloud_ang = CLOUD_R / dist
    return cloud_ang >= relax*target_ang

def merge_intervals(intv):
    if not intv: return []
    A = sorted(intv)
    cur = [A[0][0], A[0][1]]
    out = []
    for s,e in A[1:]:
        if s <= cur[1]:
            cur[1] = max(cur[1], e)
        else:
            out.append((float(cur[0]), float(cur[1])))
            cur = [s,e]
    out.append((float(cur[0]), float(cur[1])))
    return out

def union_length(intv):
    return sum(e - s for s, e in merge_intervals(intv))

def solve_quadratic(a,b,c):
    if abs(a) < 1e-12:
        if abs(b) < 1e-12: return []
        return [-c/b]
    D = b*b - 4*a*c
    if D < -1e-12: return []
    if D < 0: D = 0.0
    rD = math.sqrt(D)
    return [(-b - rD)/(2*a), (-b + rD)/(2*a)]

def ring_line_lambdas(Uxy, Mxy, dxy, R):
    # ||(Mxy + λ dxy) - Uxy||^2 = R^2
    w = Mxy - Uxy
    a = float(np.dot(dxy, dxy))
    b = 2.0 * float(np.dot(dxy, w))
    c = float(np.dot(w, w)) - R*R
    roots = solve_quadratic(a,b,c)
    return [lam for lam in roots if 0.0 <= lam <= 1.0]

def analytic_lambda_candidates(uav_name, mname, te):
    U0 = UAVS[uav_name]
    Uxy = U0[:2]
    M = missile_pos(mname, te)
    Mxy = M[:2]
    dxy = (TARGET_CENTER + np.array([0,0,5.0]) - M)[:2]
    if np.linalg.norm(dxy) < 1e-9: return []
    rmin = SPEED_MIN * te
    rmax = SPEED_MAX * te
    roots = []
    for R in (rmin, rmax):
        roots += ring_line_lambdas(Uxy, Mxy, dxy, R)
    roots += [0.0, 1.0]
    roots = sorted(set(max(0.0, min(1.0, float(x))) for x in roots))
    out = []
    for i in range(len(roots)-1):
        lam1, lam2 = roots[i], roots[i+1]

```

```

        mid = 0.5*(lam1+lam2)
        Cxy = Mxy + mid * dxy
        if rmin - 1e-6 <= np.linalg.norm(Cxy - Uxy) <= rmax + 1e-6:
            out.append(float(np.clip(mid, 0.02, 0.98)))
    # 也保留边界根 (裁剪)
    for lam in ring_line_lambdas(Uxy, Mxy, dxy, rmin) + ring_line_lambdas(Uxy, Mxy, dxy, rmax):
        lamc = float(np.clip(lam, 0.02, 0.98))
        if 0.0 <= lamc <= 1.0: out.append(lamc)
    return sorted(set(out))

def cloud_center_on_LOS(mname, te, lam):
    M = missile_pos(mname, te)
    Taxis = TARGET_CENTER + np.array([0,0,5.0])
    return M + lam * (Taxis - M)

def plan_controls_for_center(uav_name, te, C):
    U0 = UAVS[uav_name]
    Uxy, Cxy = U0[:2], C[:2]
    dxy = Cxy - Uxy
    dist = np.linalg.norm(dxy)
    v = dist / te if te > 1e-12 else float("inf")
    if not (SPEED_MIN <= v <= SPEED_MAX): return None
    if C[2] > U0[2]: return None # 只允许下抛
    fall2 = 2.0*(U0[2] - C[2]) / g
    if fall2 < 0: return None
    fall = math.sqrt(fall2)
    if te < fall: return None
    r = te - fall
    delay = 0.0 if r >= 0 else -r
    r = max(0.0, r)
    alpha = math.atan2(dxy[1], dxy[0]) # 航向角
    return dict(speed=v, heading=alpha, release=r, delay=delay)

def single_cloud_intervals_for_missile(mname, te, C0, dt=VERIFY_DT):
    # 该云对指定导弹的遮蔽区间 (20s 有效 & 下沉)
    times = np.arange(te, te + CLOUD_T + 1e-9, dt)
    occ = []
    start = None
    for i, t in enumerate(times):
        M = missile_pos(mname, t)
        Cz = C0.copy()
        Cz[2] = C0[2] - CLOUD_SINK*(t - te)

        ok = True
        for P in CYL_SAMPLES:
            if not seg_sphere_intersect(M, P, Cz, CLOUD_R):
                ok = False; break
        if ok and start is None:
            start = t
        if (i == len(times)-1 and ok) or (not ok and start is not None):
            end = t
            if end - start >= dt * 0.5:
                occ.append((float(start), float(end)))
            start = None
    return merge_intervals(occ)

@dataclass
class Candidate:
    uav: str

```

```

missile: str
te: float
lam: float
cx: float; cy: float; cz: float
speed: float
heading: float
release: float
delay: float
intervals: tuple
score: float

def generate_pair_candidates(uav, mname, dt=VERIFY_DT,
                             t0=EXPLODE_T0, step=EXPLODE_D,
topk=TOPK_PER_PAIR):
    tmax = np.linalg.norm(MISSILES[mname] - FAKE) / MISSILE_V
    allc = []
    for te in np.arange(t0, tmax, step):
        for lam in analytic_lambda_candidates(uav, mname, te):
            C = cloud_center_on_LOS(mname, te, lam)
            # 快速张角门
            if not angle_gate(mname, te, C): continue
            ctl = plan_controls_for_center(uav, te, C)
            if ctl is None: continue
            # 单云验证
            iv = single_cloud_intervals_for_missile(mname, te, C, dt=dt)
            if not iv: continue
            sc = union_length(iv)
            allc.append(Candidate(
                uav=uav, missile=mname, te=float(te), lam=float(lam),
                cx=float(C[0]), cy=float(C[1]), cz=float(C[2]),
                speed=float(ctl["speed"]), heading=float(ctl["heading"]),
                release=float(ctl["release"]), delay=float(ctl["delay"]),
                intervals=tuple(iv), score=float(sc)
            ))
    allc.sort(key=lambda c: c.score, reverse=True)
    return allc[:topk]

def cluster_key(c: Candidate):
    # 同航向同速度: 用 1° / 1m/s 的“桶”聚类
    hdeg = math.degrees(c.heading)
    hbin = round(hdeg / HEADING_BIN_DEG) * HEADING_BIN_DEG
    vbin = round(c.speed / SPEED_BIN_MPS) * SPEED_BIN_MPS
    return (round(hbin, 3), round(vbin, 3))

def obey_release_gaps(events, min_gap=MIN_RELEASE_GAP):
    R = sorted(e.release for e in events)
    for i in range(1, len(R)):
        if R[i] - R[i-1] < min_gap - 1e-12:
            return False
    return True

def is_fully_occluded_at(mname, t, chosen_events):
    M = missile_pos(mname, t)
    for P in CYL_SAMPLES:
        covered = False
        for e in chosen_events:
            if not (e.te <= t <= e.te + CLOUD_T):
                continue
            C = np.array([e.cx, e.cy, e.cz - CLOUD_SINK*(t - e.te)])
            if seg_sphere_intersect(M, P, C, CLOUD_R):

```

```

        covered = True
        break
    if not covered:
        return False
    return True

def per_missile_union_intervals(chosen_events, dt=VERIFY_DT):
    out = {}
    for m in MISSILES.keys():
        tmax = np.linalg.norm(MISSILES[m] - FAKE) / MISSILE_V
        times = np.arange(0.0, tmax + 1e-9, dt)
        occ = []
        start = None
        for i, t in enumerate(times):
            ok = is_fully_occluded_at(m, t, chosen_events)
            if ok and start is None: start = t
            if (i == len(times)-1 and ok) or (not ok and start is not
None):
                end = t
                if end - start >= dt*0.5: occ.append((float(start),
float(end)))
                start = None
        out[m] = merge_intervals(occ)
    return out

def objective_sum_of_unions(chosen_events, dt=VERIFY_DT):
    per = per_missile_union_intervals(chosen_events, dt=dt)
    return sum(union_length(per[m]) for m in per.keys()), per

# 主流程

def solve_problem5(verbose=True):
    tic = time.time()
    if verbose:
        print("=== 问题 5: 启发式 (解析候选→聚类→簇内≤3→全局贪心) ===")

    # 1) 生成 (UAV, Missile) 候选
    pair_cands = collections.defaultdict(list)
    total_pairs = 0
    for uav in UAVS.keys():
        for m in MISSILES.keys():
            C = generate_pair_candidates(uav, m)
            pair_cands[(uav,m)] = C
            total_pairs += len(C)
            if verbose:
                print(f"[{uav} × {m}] 候选 = {len(C)}")
    if verbose:
        print(f"候选总数 (去重前) ≈ {total_pairs}")

    # 2) 聚类: 按 UAV 合并其跨三枚导弹的候选后, 按同航向同速度分簇
    uav_clusters = {}
    for uav in UAVS.keys():
        lst = []
        for m in MISSILES.keys():
            lst += pair_cands[(uav,m)]
        dic = collections.defaultdict(list)
        for c in lst:

```

```

        dic[cluster_key(c)].append(c)
# 每个簇内部按单云分数排序
for k in dic.keys():
    dic[k].sort(key=lambda c: c.score, reverse=True)
uav_clusters[uav] = dic
if verbose:
    print(f"[{uav}] 簇数 = {len(dic)} (同航向同速度)")

# 3) 对每个簇, 枚举  $\leq 3$  的“簇内组合” (先裁 top-L) 并记录
cluster_bundle = {}
for uav, dic in uav_clusters.items():
    cluster_bundle[uav] = {}
    for key, clist in dic.items():
        base = clist[:CLUSTER_ENUM_TOP_L]
        combos = [[]]
        for k in (1, 2, 3):
            combos += [list(x) for x in itertools.combinations(base, k)]

        good = []
        for xs in combos:
            if len(xs) > MAX_SHOTS_PER_UAV: continue
            xs_sorted = sorted(xs, key=lambda e: e.release)
            if obey_release_gaps(xs_sorted, MIN_RELEASE_GAP):
                good.append(xs_sorted)
        cluster_bundle[uav][key] = good

# 4) 按 UAV 逐个贪心选簇
chosen_by_uav = {uav: [] for uav in UAVS.keys()}
chosen_all = []
for uav in UAVS.keys():
    best_gain = -1.0
    best_key = None
    best_set = []

    base_score, _ = objective_sum_of_unions(chosen_all)
    # 遍历该 UAV 所有簇
    for key, bundles in cluster_bundle[uav].items():
        for xs in bundles:
            news = chosen_all + xs
            sc, _ = objective_sum_of_unions(news)
            gain = sc - base_score
            if gain > best_gain:
                best_gain, best_key, best_set = gain, key, xs
    if best_gain <= 1e-9:
        if verbose:
            print(f"[{uav}] 选择空簇 (无正增益)")
        continue
    chosen_by_uav[uav] = best_set
    chosen_all += best_set
    if verbose:
        hbin, vbin = best_key
        print(f"[{uav}] 选中簇 (heading~{hbin}°, speed~{vbin} m/s), 弹数={len(best_set)}, 增益+{best_gain:.3f}s")

# 5) 终评估 + 导出
total, per = objective_sum_of_unions(chosen_all)
if verbose:
    print("\n=== 终评估 ===")
    for m in MISSILES.keys():

```

```

        print(f"{m}: {union_length(per[m]):.3f} s   intervals={per[m]}")
    print(f"SUM = {total:.3f} s")

# 导出 result3.xlsx
rows = []
for uav in UAVS.keys():
    es = chosen_by_uav[uav]
    if not es:
        continue
    hdeg = round(math.degrees(es[0].heading), 4)
    vmgs = round(es[0].speed, 4)
    for e in es:
        rows.append(dict(
            UAV=uav, Missile=e.missile,
            heading_deg=hdeg, speed_mps=vmgs,
            release_time_s=round(e.release, 4),
            explode_time_s=round(e.te, 4),
            delay_s=round(e.delay, 4),
            explode_x=round(e.cx, 3),
            explode_y=round(e.cy, 3),
            explode_z=round(e.cz, 3),
            single_cloud_intervals=str(list(e.intervals))
        ))
rows2 = []
for m in MISSILES.keys():
    for (s,e) in per[m]:
        rows2.append(dict(Missile=m, t_start=s, t_end=e, length=e-s))

with pd.ExcelWriter(RESULT_XLSX, engine="openpyxl") as w:
    pd.DataFrame(rows).to_excel(w, index=False, sheet_name="result3_events")
    pd.DataFrame(rows2).to_excel(w, index=False, sheet_name="per_missile_unions")

toc = time.time()
if verbose:
    print(f"已保存: {RESULT_XLSX}")
    print(f"Runtime: {toc - tic:.2f} s")
return dict(
    chosen_by_uav=chosen_by_uav,
    chosen_all=chosen_all,
    per_missile_intervals=per,
    total=total
)

if __name__ == "__main__":
    solve_problem5(verbose=True)

```

代码文件 8 第五问粒子群优化阶段(p5ada.py)

```

import numpy as np
import pandas as pd
import math, time, random
from dataclasses import dataclass, field
from typing import Dict, List, Tuple
from tqdm import tqdm
from problem_1 import CInterferenceModel

FILE = "result3.xlsx"

```

```

SHEET = "result3_events"
OUT_XLSX = "result_pso.xlsx"

DT_EVAL      = 0.2
RIM_POINTS_N = 64
V_MIN, V_MAX = 70.0, 140.0
HEADING_WRAP = 360.0
MIN_GAP_RELEASE = 0.0
MIN_GAP_EXPLODE = 0.0

N_PART  = 100
N_ITER  = 200
W_INERT = 0.65
C_COG   = 1.6
C_SOC   = 1.6
SEED_JITTER_HEADING = 5.0
SEED_JITTER_SPEED   = 6.0
SEED_JITTER_REL      = 0.6
SEED_JITTER_DELAY    = 0.6
np.random.seed(42)
random.seed(42)

MISSILES0 = {
    "M1": np.array([20000.0, 0.0, 2000.0]),
    "M2": np.array([19000.0, 600.0, 2100.0]),
    "M3": np.array([18000.0, -600.0, 1900.0]),
}
FAKE = np.array([0.0, 0.0, 0.0])
UAVS0 = {
    "FY1": np.array([17800.0, 0.0, 1800.0]),
    "FY2": np.array([12000.0, 1400.0, 1400.0]),
    "FY3": np.array([ 6000.0, -3000.0, 700.0]),
    "FY4": np.array([11000.0, 2000.0, 1800.0]),
    "FY5": np.array([13000.0, -2000.0, 1300.0]),
}

@dataclass
class Event:
    uav: str
    idx: int
    release: float
    delay: float

@dataclass
class UAVGroup:
    uav: str
    heading_deg: float
    speed_mps: float
    events: List[Event] = field(default_factory=list)

@dataclass
class ProblemData:
    groups: Dict[str, UAVGroup]
    t_max: float

def wrap_deg(a: float) -> float:
    a = np.mod(a, HEADING_WRAP)
    if a < 0: a += HEADING_WRAP
    return float(a)

```

```

def build_from_excel(path: str, sheet: str) -> ProblemData:
    df = pd.read_excel(path, sheet_name=sheet)
    need = ["UAV", "heading_deg", "speed_mps", "release_time_s", "explode_time_s"]
    for c in need:
        if c not in df.columns:
            raise ValueError(f"缺少列: {c}")
    df = df.sort_values(["UAV", "explode_time_s", "release_time_s"]).reset_index(drop=True)
    df["bomb_idx"] = df.groupby("UAV").cumcount() + 1

    # 基于已搜索解, 聚合"同 UAV 统一 heading/speed"
    groups: Dict[str, UAVGroup] = {}
    for uav, sub in df.groupby("UAV"):
        heading0 = float(np.mean(sub["heading_deg"]))
        speed0 = float(np.mean(sub["speed_mps"]))
        g = UAVGroup(uav=uav, heading_deg=wrap_deg(heading0),
                      speed_mps=float(np.clip(speed0, V_MIN, V_MAX)))
        for r in sub.itertuples():
            delay = float(r.explode_time_s - r.release_time_s)
            if delay < 0:
                raise ValueError(f"{uav} 存在 explode_time_s < release_time_s")
            g.events.append(Event(uav=uav, idx=int(r.bomb_idx),
                                 release=float(r.release_time_s),
                                 delay=delay))

        groups[uav] = g

    # 时间上限: 导弹到达假目标时间 与 所有云有效末端 的最大值
    tmp_model = CInterferenceModel()
    tmp_model.fake_target = FAKE
    missile_v = tmp_model.m_speed
    T_missiles = [ np.linalg.norm(MISSILES0[m]-FAKE)/missile_v for m in ["M1", "M2", "M3"] ]
    T_clouds = []
    for g in groups.values():
        for e in g.events:
            T_clouds.append(e.release + e.delay + tmp_model.effective_time)
    t_max = max(max(T_missiles), max(T_clouds) if T_clouds else 0.0)
    # 稍微加个缓冲
    t_max += 2.0
    return ProblemData(groups=groups, t_max=float(t_max))

def problem_to_vector(pb: ProblemData) -> Tuple[np.ndarray, List[Tuple[str, str, int]]]:
    xs = []
    info = []
    for uav in sorted(pb.groups.keys()):
        g = pb.groups[uav]
        xs += [g.heading_deg, g.speed_mps]
        info.append((uav, "heading", -1))
        info.append((uav, "speed", -1))
    for uav in sorted(pb.groups.keys()):
        g = pb.groups[uav]
        for e in g.events:
            xs += [e.release, e.delay]
            info.append((uav, "release", e.idx))

```

```

        info.append((uav, "delay", e.idx))
    return np.array(xs, dtype=float), info

def vector_to_problem(x: np.ndarray, info: List[Tuple[str, str, int]],
base: ProblemData) -> ProblemData:
    groups: Dict[str, UAVGroup] = {}
    for uav, g in base.groups.items():
        groups[uav] = UAVGroup(
            uav=uav,
            heading_deg=g.heading_deg,
            speed_mps=g.speed_mps,
            events=[Event(uav=e.uav, idx=e.idx, release=e.release, de-
lay=e.delay) for e in g.events]
        )
    for val, (uav, key, idx) in zip(x.tolist(), info):
        g = groups[uav]
        if key == "heading":
            g.heading_deg = wrap_deg(val)
        elif key == "speed":
            g.speed_mps = float(np.clip(val, V_MIN, V_MAX))
        elif key in ("release", "delay"):
            ev = next(e for e in g.events if e.idx == idx)
            if key == "release":
                ev.release = float(max(0.0, val))
            else:
                ev.delay = float(max(0.0, val))

    # 最小间隔 (可选)
    if MIN_GAP_RELEASE > 0 or MIN_GAP_EXPLODE > 0:
        for g in groups.values():
            g.events.sort(key=lambda e: e.release)
            for i in range(1, len(g.events)):
                prev, cur = g.events[i-1], g.events[i]
                if MIN_GAP_RELEASE > 0 and cur.release < prev.release +
MIN_GAP_RELEASE:
                    cur.release = prev.release + MIN_GAP_RELEASE
                if MIN_GAP_EXPLODE > 0:
                    prev_e = prev.release + prev.delay
                    cur_e = cur.release + cur.delay
                    if cur_e < prev_e + MIN_GAP_EXPLODE:
                        shift = (prev_e + MIN_GAP_EXPLODE) - cur_e
                        cur.release += shift
    return ProblemData(groups=groups, t_max=base.t_max)

def evaluate_total_occlusion(pb: ProblemData, dt=DT_EVAL,
rim_points=RIM_POINTS_N) -> Tuple[float, Dict[str, List[Tu-
ple[float, float]]]]:

    model = CInterferenceModel()

    model.fake_target = FAKE
    target_points = model.get_true_points(num_points=rim_points)

    cloud_funcs = []
    for uav, g in pb.groups.items():
        model.FYl_init = UAVS0[uav]
        model.f_angle = g.heading_deg
        for e in g.events:
            f = model.calculate_c_center(
                explode_time=e.release + e.delay,

```

```

        release_time=e.release,
        drone_speed=g.speed_mps
    )
    cloud_funcs.append(f)

# 时间网格
T = pb.t_max
times = np.arange(0.0, T + 1e-9, dt)

def mask_to_intervals(mask: np.ndarray) -> List[Tuple[float, float]]:
    ivs=[]; on=False
    for i,b in enumerate(mask):
        if b and not on:
            st = times[i]; on=True
            if on and (i==len(mask)-1 or not mask[i+1]):
                ed = times[i] + (times[1]-times[0] if len(times)>1 else 0.0)
                ivs.append((st, ed)); on=False
    return ivs

intervals_by_missile: Dict[str, List[Tuple[float, float]]] = {}
total = 0.0

for M in ["M1", "M2", "M3"]:
    model.M1_init = MISSILES0[M]
    mask = np.zeros(len(times), dtype=bool)
    for i,t in enumerate(times):
        missile_pos = model.calculate_m_trajectory(t)
        ok_union = False
        # 并集: 存在某朵云单独能完全遮蔽即可
        for f in cloud_funcs:
            C = f(t)
            if model.check_effective_shielding(missile_pos, C, target_points):
                ok_union = True
                break
        mask[i] = ok_union
    ivs = mask_to_intervals(mask)
    intervals_by_missile[M] = ivs
    total += sum(b-a for a,b in ivs)

return total, intervals_by_missile

def pso_optimize(seed_pb: ProblemData):
    x0, info = problem_to_vector(seed_pb)

    # 初始粒子群
    D = len(x0)
    X = np.zeros((N_PART, D), dtype=float)
    V = np.zeros((N_PART, D), dtype=float)

    X[0] = x0.copy()
    for k in range(1, N_PART):
        x = x0.copy()
        for i, (uav, key, idx) in enumerate(info):
            if key == "heading":
                x[i] += np.random.normal(0, SEED_JITTER_HEADING)
            elif key == "speed":
                x[i] += np.random.normal(0, SEED_JITTER_SPEED)
            elif key == "release":
                x[i] += np.random.normal(0, SEED_JITTER_REL)

```

```

        elif key == "delay":
            x[i] += np.random.normal(0, SEED_JITTER_DELAY)
        X[k] = x

    def fval(xvec):
        pb = vector_to_problem(xvec, info, seed_pb)
        tot, _ = evaluate_total_occlusion(pb, dt=DT_EVAL,
rim_points=RIM_POINTS_N)
        return -tot

    # 评估初值
    F = np.array([fval(X[k]) for k in range(N_PART)], dtype=float)
    Pbest = X.copy()
    Fp = F.copy()
    g_idx = int(np.argmin(F))
    Gbest = X[g_idx].copy()
    Fg = F[g_idx] # 当前全局最小 (= -最大覆盖)

    # 进度条
    pbar = tqdm(range(1, N_ITER+1), ncols=100, desc="PSO")
    pbar.set_postfix(best_total=f"{-Fg:.3f}s")

    for it in pbar:
        r1, r2 = np.random.rand(N_PART, D), np.random.rand(N_PART, D)
        V = W_INERT*V + C_COG*r1*(Pbest - X) + C_SOC*r2*(Gbest - X)
        X = X + V

        # 投影到可行域
        for k in range(N_PART):
            x = X[k]
            for i, (uav, key, idx) in enumerate(info):
                if key == "heading":
                    x[i] = wrap_deg(x[i])
                elif key == "speed":
                    x[i] = float(np.clip(x[i], V_MIN, V_MAX))
                elif key == "release":
                    x[i] = float(max(0.0, x[i]))
                elif key == "delay":
                    x[i] = float(max(0.0, x[i]))
            X[k] = x

        # 评估
        F = np.array([fval(X[k]) for k in range(N_PART)], dtype=float)

        improved = F < Fp
        Pbest[improved] = X[improved]
        Fp[improved] = F[improved]
        g_idx = int(np.argmin(Fp))
        if Fp[g_idx] < Fg:
            Gbest = Pbest[g_idx].copy()
            Fg = Fp[g_idx]

        # 动态显示“当前最优”
        pbar.set_postfix(best_total=f"{-Fg:.3f}s", iter=it)

    # 输出最优
    best_pb = vector_to_problem(Gbest, info, seed_pb)
    best_total, best_iv = evaluate_total_occlusion(best_pb, dt=max(0.1,
DT_EVAL/2), rim_points=max(64, RIM_POINTS_N))

```

```

print("\n=== PSO 结果 ===")
print(f"总并集完全遮蔽: {best_total:.3f} s")
for M in ["M1", "M2", "M3"]:
    tM = sum(b-a for a,b in best_iv[M])
    print(f"{M}: {tM:.3f} s  区间: {best_iv[M]}")

# 导出表格
rows = []
for uav in sorted(best_pb.groups.keys()):
    g = best_pb.groups[uav]
    for e in g.events:
        rows.append({
            "UAV": uav,
            "heading_deg": g.heading_deg,
            "speed_mps": g.speed_mps,
            "release_time_s": e.release,
            "explode_time_s": e.release + e.delay,
            "bomb_idx": e.idx
        })
out_df = (pd.DataFrame(rows)
          .sort_values(["UAV", "explode_time_s", "re-
lease_time_s", "bomb_idx"])
          .reset_index(drop=True))
with pd.ExcelWriter(OUT_XLSX) as w:
    out_df.to_excel(w, sheet_name="pso_events", index=False)
print(f"已保存: {OUT_XLSX} (sheet=pso_events)")

return best_pb, best_total, best_iv

def solve_problem_5():
    t0 = time.time()
    seed_pb = build_from_excel(FILE, SHEET)
    best_pb, best_total, best_iv = pso_optimize(seed_pb)
    print(f"Runtime: {time.time()-t0:.1f} s")

if __name__ == "__main__":
    solve_problem_5()

```